



Challenge BinaryGecko

Eugenio D. Deluca

(KarasuRØØT) – Date: 01/10/2025

Índice

Nivel 0 (Intro).....	1
Herramientas:	1
Nivel 1 (Static Key).....	2
Nivel 2 (Key generation).....	7
Nivel 3 (XOR decryption).....	12
Nivel 4 (Math Operations).....	17
Nivel 5 (Unique Key)	19
Nivel 6 (Licence file).....	22
Nivel 7 (Obsucated code).....	26
Conclusión final	38
Scripts	39
Challenge2-BinaryGeckoENG.py	39
Challenge4-BinaryGeckoENG.py	40
Challenge5-BinaryGeckoENG.py	42
Challenge6-BinaryGeckoENG.py	43

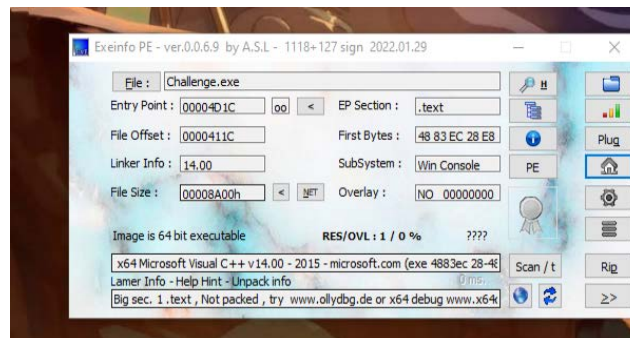
Nivel 0 (Intro)

El presente documento detalla la metodología y los hallazgos obtenidos durante la resolución del desafío de Ingeniería Inversa 'Challenge BinaryGecko'.

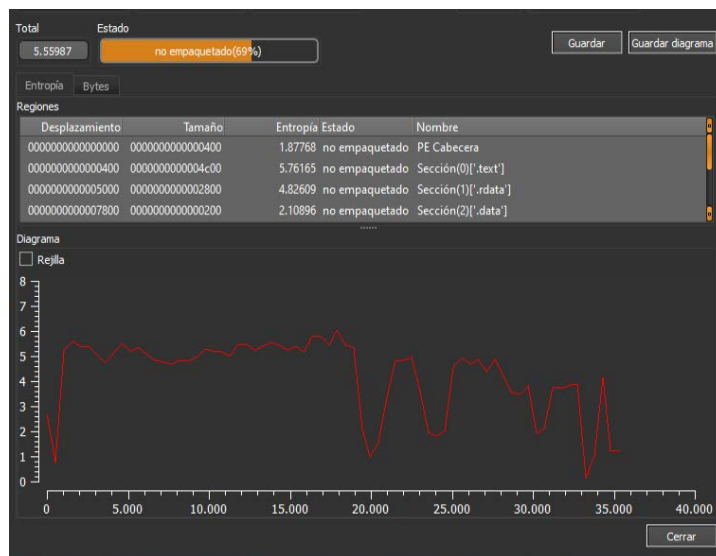
Herramientas:

- IDA PRO
- .x64
- Ingenio

Como indicaban las instrucciones cambié la extensión de “Challenge.txt” por “Challenge.exe” – Luego por curiosidad abrí el Exeinfo PE – para saber a lo que me enfrentaba.



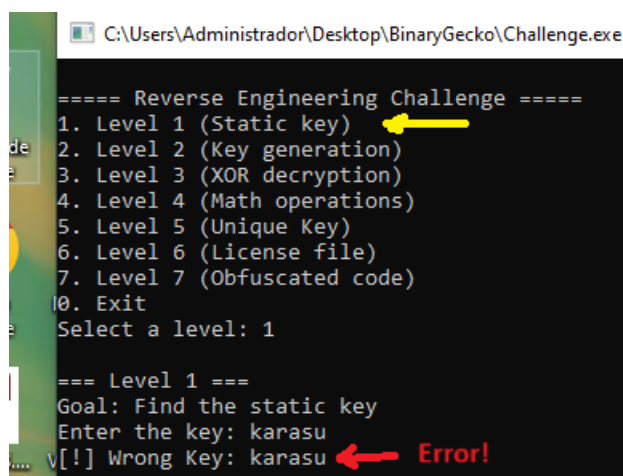
Como se ve en esa imagen, es un ejecutable de 64 bits desarrollado en C/C++ (mala noticia para mí cuando no tengo tanta práctica en ejecutables de 64 bits – pero no es problema).



Y una muy buena noticia para mí no está empaquetado esta vez.

Nivel 1 (Static Key)

Bien luego de esta pequeño “intro” comenzamos con el Nivel 1 (Static key)



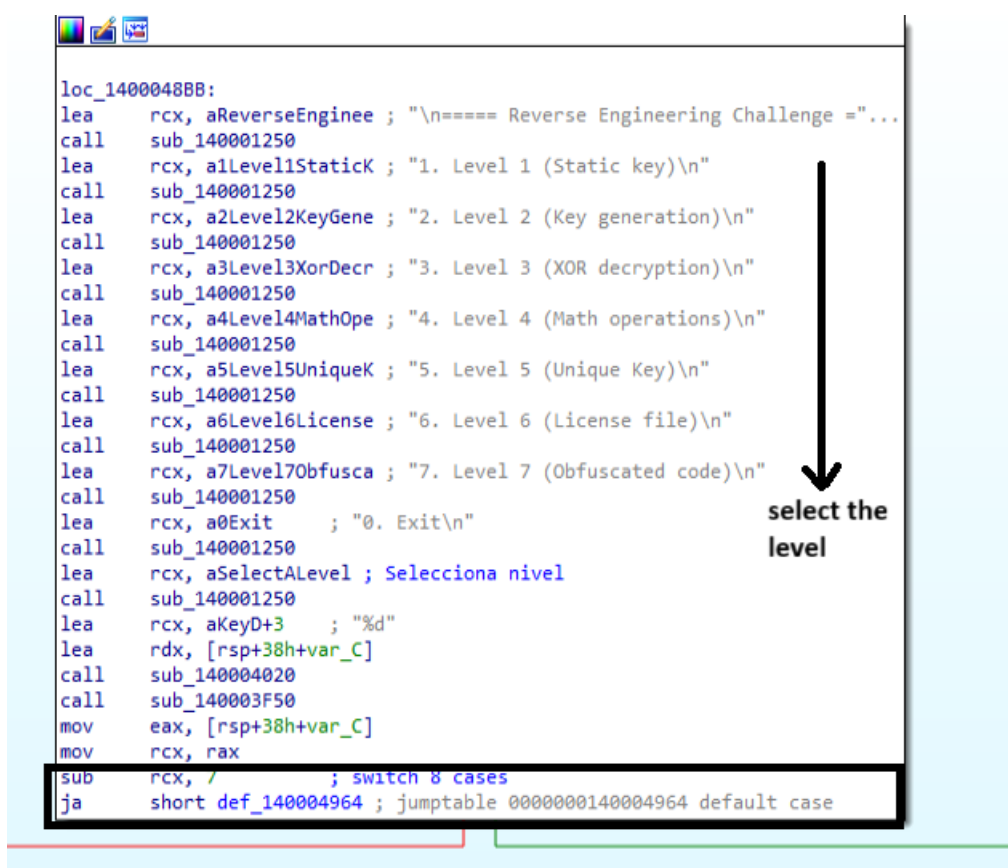
```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 1

=== Level 1 ===
Goal: Find the static key
Enter the key: karasu
v[!] Wrong Key: karasu Error!
```

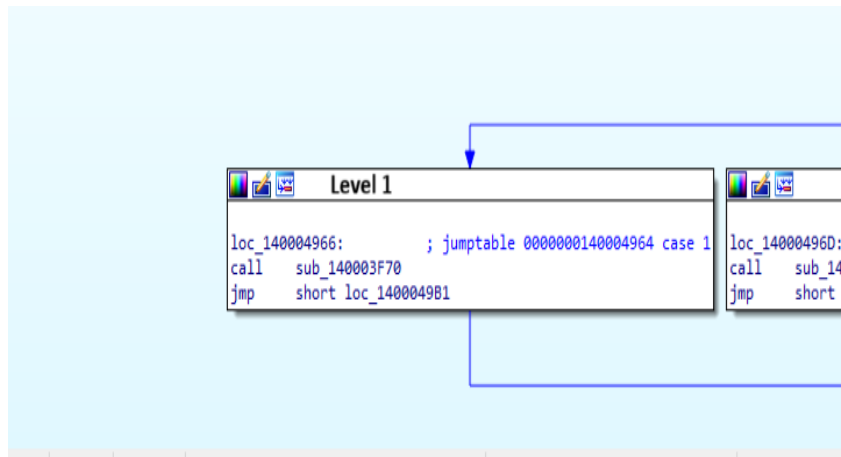
Para hacer una prueba rápida, indique una llave a modo de prueba (“karasu”) y por supuesto, indica que está equivocado.

Comienzo abriendo IDA PRO:

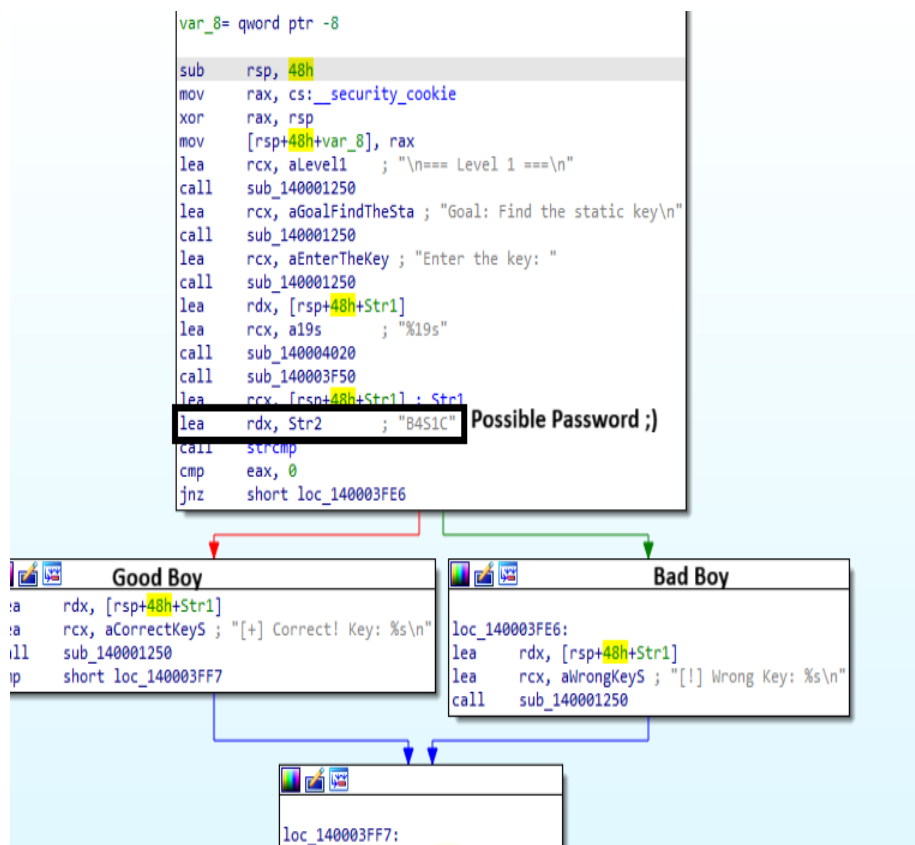


```
loc_1400048BB:
lea rcx, aReverseEnginee ; "\n===== Reverse Engineering Challenge ="...
call sub_140001250
lea rcx, a1Level1StaticK ; "1. Level 1 (Static key)\n"
call sub_140001250
lea rcx, a2Level2KeyGene ; "2. Level 2 (Key generation)\n"
call sub_140001250
lea rcx, a3Level3XorDecr ; "3. Level 3 (XOR decryption)\n"
call sub_140001250
lea rcx, a4Level4MathOpe ; "4. Level 4 (Math operations)\n"
call sub_140001250
lea rcx, a5Level5UniqueK ; "5. Level 5 (Unique Key)\n"
call sub_140001250
lea rcx, a6Level6License ; "6. Level 6 (License file)\n"
call sub_140001250
lea rcx, a7Level7Obfusca ; "7. Level 7 (Obfuscated code)\n"
call sub_140001250
lea rcx, a0Exit ; "0. Exit\n"
call sub_140001250
lea rcx, aSelectAlevel ; Selecciona nivel
call sub_140001250
lea rcx, aKeyD+3 ; "%d"
lea rdx, [rsp+38h+var_C]
call sub_140004020
call sub_140003F50
mov eax, [rsp+38h+var_C]
mov rcx, rax
sub rcx, / ; switch 8 cases
ja short def_140004964 ; jumtable 00000000140004964 default case
```

Por supuesto, este análisis será realizado por primera y única vez ahora. Esto implica que como sabemos es un programa con la sentencia “case” el cual dependiendo de numero indiquemos, es donde saltará.



Como podremos ver, así es para los siete niveles de dificultad de este reto, por cada nivel, existe un case y luego una llamada a la función correspondiente. Por ello para este primer nivel ingreso en “call sub_140003F70”.



En este primer nivel, podemos diferenciar claramente a “Chico Bueno” y “Chico Malo” y adicionalmente, la posible contraseña “B4S1C” está hardcoded.

```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

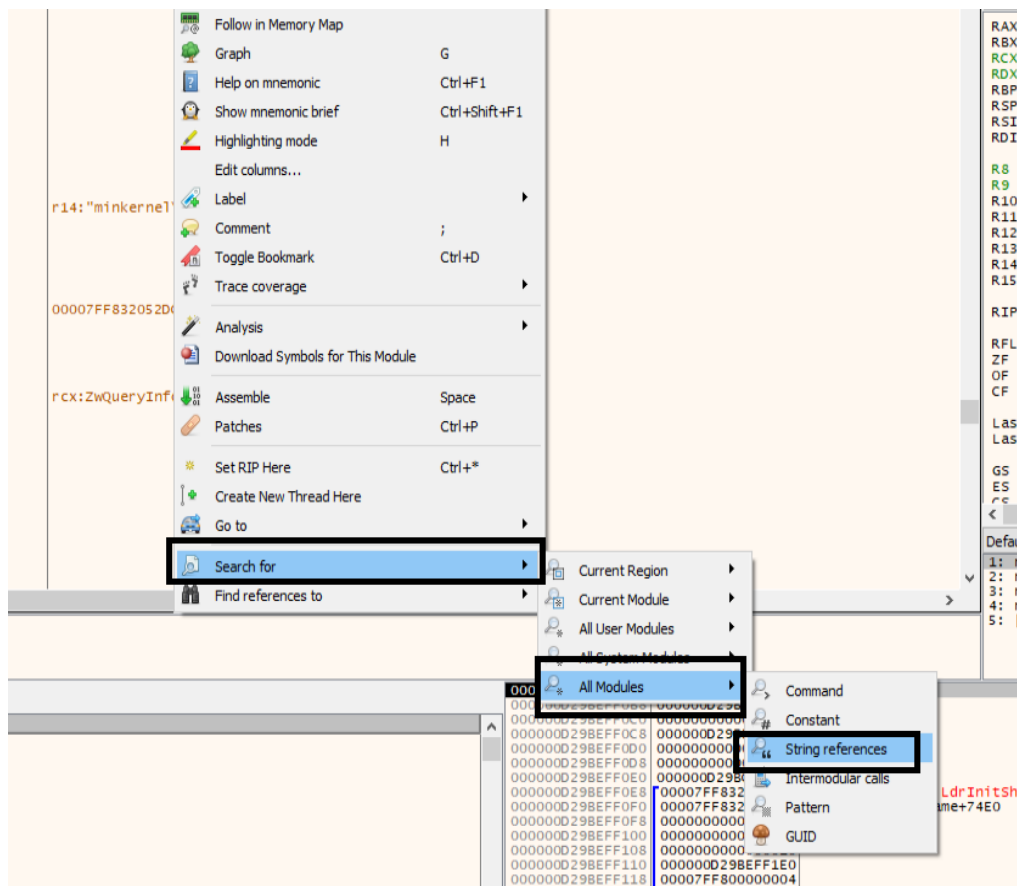
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 1

=== Level 1 ===
Goal: Find the static key
Enter the key: B451C
[+] Correct! Key: B451C Password found! First level passed!

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: _
```

Perfecto, el primer nivel fue superado con éxito – Entonces, con solo revisar el flujo del programa en el IDA PRO, lo encontramos sin problemas.

Como un adicional, en x64 también se encuentra, sencillamente buscando las cadenas – click derecho -> Search for -> All Modules -> Strings references -



00007FF7CA4328D4	lea rcx,qword ptr ds:[7FF7CA4363FF]	00007FF7CA4363FF	"string too long"
00007FF7CA432C9F	lea rdx,qword ptr ds:[7FF7CA4363EA]	00007FF7CA4363EA	"bad array new leng
00007FF7CA432DA6	lea rax,qword ptr ds:[7FF7CA4363A2]	00007FF7CA4363A2	"Unknown exception"
00007FF7CA433E74	lea rcx,qword ptr ds:[7FF7CA4363B4]	00007FF7CA4363B4	"invalid string pos
00007FF7CA433F83	lea rcx,qword ptr ds:[7FF7CA43680E]	00007FF7CA43680E	"n== Level 1 =="
00007FF7CA433F8F	lea rcx,qword ptr ds:[7FF7CA436616]	00007FF7CA436616	"Goal: Find the sta
00007FF7CA433F9B	lea rcx,qword ptr ds:[7FF7CA43650A]	00007FF7CA43650A	"Enter the key: "
00007FF7CA433FAC	lea rcx,qword ptr ds:[7FF7CA436393]	00007FF7CA436393	"%19s"
00007FF7CA433FC2	lea rdx,qword ptr ds:[7FF7CA436442]	00007FF7CA436442	"B451C"
00007FF7CA433FD8	lea rcx,qword ptr ds:[7FF7CA4366F5]	00007FF7CA4366F5	"[+] Correct! Key:
00007FF7CA433FE8	lea rcx,qword ptr ds:[7FF7CA4366E2]	00007FF7CA4366E2	"[!] Wrong Key: %s\
00007FF7CA433	challenge,00007FF7CA433FC2 00007FF7CA436442:"B451C"	00007FF7CA4367FC	"n== Level 2 =="
00007FF7CA433	lea rdx,qword ptr ds:[7FF7CA436442]	00007FF7CA436940	"Goal: Get the corr
00007FF7CA433	call <JMP.&strcmp>	00007FF7CA43656F	"Enter your name: "
00007FF7CA433	cmp eax,0	00007FF7CA436393	"%19s"
00007FF7CA433	jne challenge.7FF7CA433FE6	00007FF7CA43651A	"Enter the generate
00007FF7CA433	lea rdx,qword ptr ss:[rsp+20]	00007FF7CA436393	"%19s"
00007FF7CA433	lea rcx,qword ptr ds:[7FF7CA4366F5]	00007FF7CA4366C2	"[+] Correct! Gener
00007FF7CA433	call challenge.7FF7CA431250	00007FF7CA436740	"[!] wrong key, try
00007FF7CA433	jmp challenge.7FF7CA433FF7	00007FF7CA436160	"\"I"
00007FF7CA433	lea rdx,qword ptr ss:[rsp+20]	00007FF7CA4367EA	"n== Level 3 =="
00007FF7CA433	lea rcx,qword ptr ds:[7FF7CA4366E2]	00007FF7CA436790	"Goal: Key is XOR-e
00007FF7CA433	call challenge.7FF7CA431250	00007FF7CA4364F4	"Enter the final ke
00007FF7CA433	mov rcx,qword ptr ss:[rsp+40]	00007FF7CA436393	"%19s"
00007FF7CA433	xor rcx,rspx	00007FF7CA436670	"real key: %s\n"
00007FF7CA433	cmp rcx,qword ptr ds:[7FF7CA439040]	00007FF7CA43669A	"[+] Congratulator
00007FF7CA433	jne challenge.7FF7CA43400F	00007FF7CA43675A	"[!] Wrong. Keep tr
00007FF7CA433	jmp challenge.7FF7CA43400A	00007FF7CA4367D8	"n== Level 4 =="
00007FF7CA433	add rsp,48	00007FF7CA43663A	"Goal: Calculate th
00007FF7CA433	ret	00007FF7CA436534	"Enter a 4-digit nu
00007FF7CA433	call challenge.7FF7CA4355F0	00007FF7CA43639E	"%4s"
00007FF7CA433	int3	00007FF7CA436422	"KEY%d"
00007FF7CA4340E	lea rcx,qword ptr ds:[7FF7CA436592]	00007FF7CA436592	"Now enter the calc
00007FF7CA4341F	lea rcx,qword ptr ds:[7FF7CA436393]	00007FF7CA436393	"%19s"
00007FF7CA43449	lea rcx,qword ptr ds:[7FF7CA43667E]	00007FF7CA43667E	"[+] Success! Valid
00007FF7CA43457	lea rcx,qword ptr ds:[7FF7CA436740]	00007FF7CA436740	"[!] wrong key, try
00007FF7CA43459	lea rcx,qword ptr ds:[7FF7CA436820]	00007FF7CA436820	"n== Level 5 (Unl
00007FF7CA434695	lea r8,qword ptr ds:[7FF7CA436438]	00007FF7CA436438	"%s-%s-REV"
00007FF7CA4346B1	lea rcx,qword ptr ds:[7FF7CA4368D8]	00007FF7CA4368D8	"[!] Debugger detec
00007FF7CA4346BD	lea rcx,qword ptr ds:[7FF7CA4365E8]	00007FF7CA4365E8	"Run without de
00007FF7CA4346CB	lea rcx,qword ptr ds:[7FF7CA43650A]	00007FF7CA43650A	"Enter the key: "
00007FF7CA4346DC	lea rcx,qword ptr ds:[7FF7CA436398]	00007FF7CA436398	"%255s"
00007FF7CA43470C	lea rcx,qword ptr ds:[7FF7CA4366F5]	00007FF7CA4366F5	"[+] Correct! Key:
00007FF7CA434722	lea rcx,qword ptr ds:[7FF7CA436708]	00007FF7CA436708	"[!] Wrong. valid k
00007FF7CA434781	lea rdx,qword ptr ds:[7FF7CA436470]	00007FF7CA436470	"BinaryGecko"
00007FF7CA43479D	lea rdx,qword ptr ds:[7FF7CA436468]	00007FF7CA436468	"8988"
00007FF7CA4347D3	lea rcx,qword ptr ds:[7FF7CA436428]	00007FF7CA436428	"licencia.enc"
00007FF7CA4347DA	lea rdx,qword ptr ds:[7FF7CA436435]	00007FF7CA436435	"rb"
00007FF7CA4347F4	lea rcx,qword ptr ds:[7FF7CA436772]	00007FF7CA436772	"Error: License key

Search: [Type here to filter results...]

ntdll.dll 100%

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused 10241 string(s) in 2437ms

Al ser algo sencillo, que el password está hardcodedo, la única opción sería parchearlo.

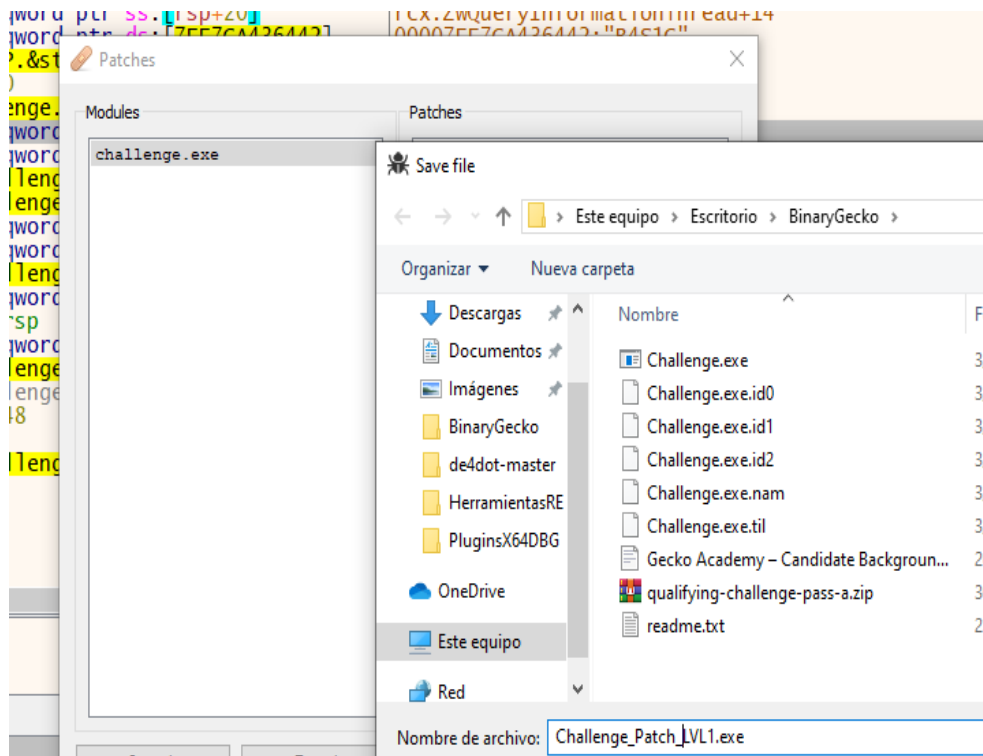
La línea a cambiar es:

00007FF7CA433FCE	83F8 00	cmp eax,0
00007FF7CA433FD1	75 13	jne challenge.7FF7CA433FE6
00007FF7CA433FD3	48:8D5424 20	lea rdx,qword ptr ss:[rsp+20]
00007FF7CA433FD8	48:8D0D 16270000	lea rcx,qword ptr ds:[7FF7CA4366F5]

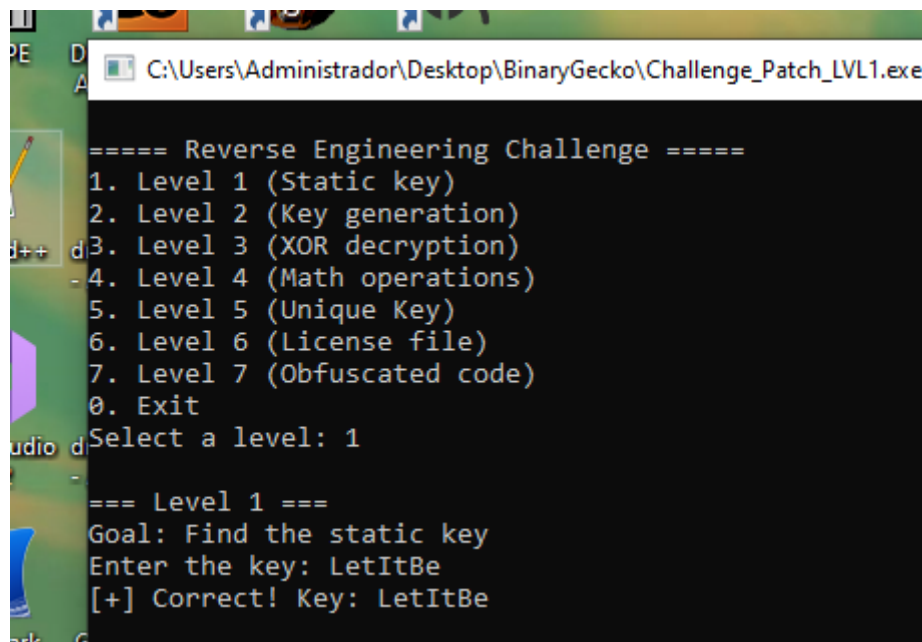
Yo decidí reemplazarlo por:

00007FF7CA433FCE	83F8 00	cmp eax,0
00007FF7CA433FD1	74 13	je challenge.7FF7CA433FE6
00007FF7CA433FD3	48:8D5424 20	lea rdx,qword ptr ss:[rsp+20]

Luego parcheo el programa:

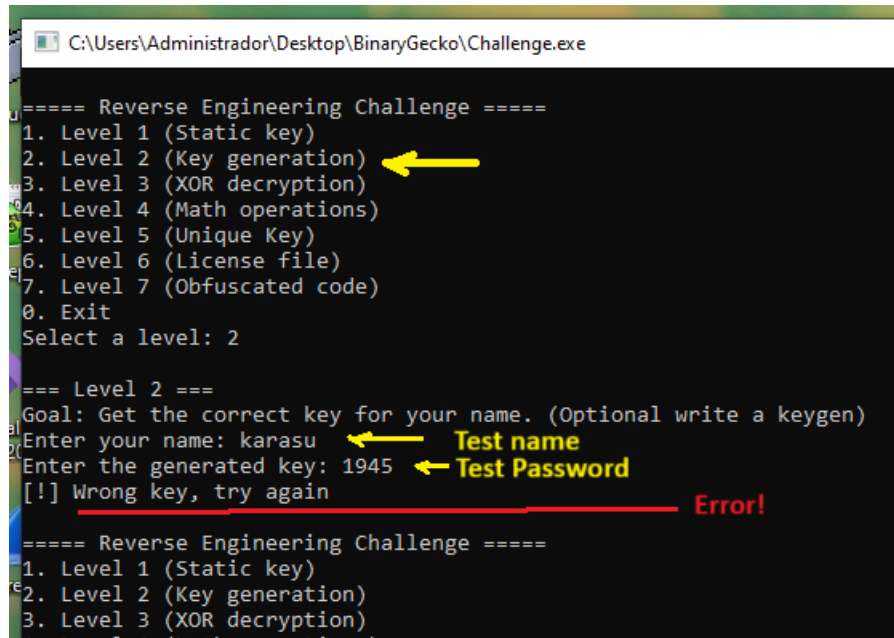


Y lo ejecuto – Uso cualquier contraseña – por ejemplo LetItBe Tambien será adjuntado el archivo “Challenge_Patch_LVL1.exe”.



Nivel 2 (Key generation)

Comencemos con el Nivel 2:

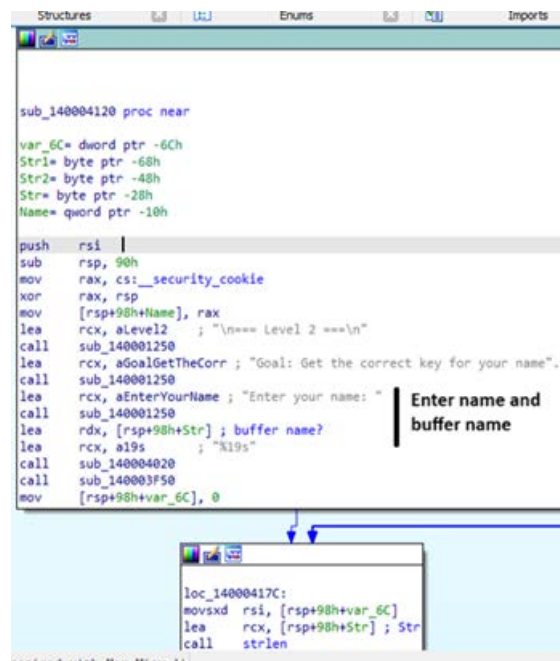


```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 2

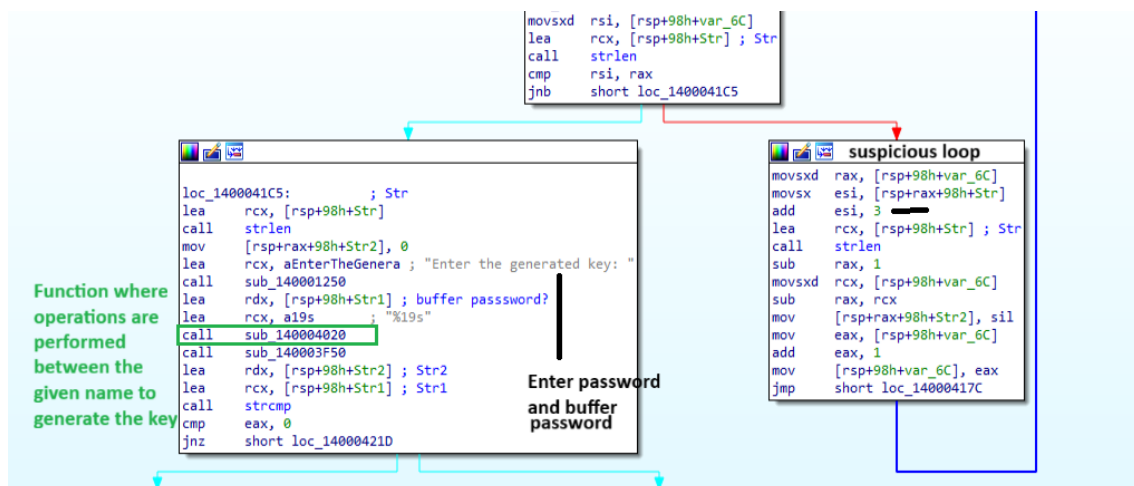
=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: karasu
Enter the generated key: 1945
[!] Wrong key, try again

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
```



Como vemos en la imagen anterior, ahí se encuentra para ingresar el “name” y más abajo está el “buffer del nombre” – y en la imagen que prosigue, por un lado, esta donde ingresamos nuestra clave y su buffer correspondiente, además de la Función donde se realizan operaciones entre el nombre dado para generar la clave.

Y luego está un loop el cual como vemos, por ejemplo, hace operaciones, así que intuyo que es el responsable de hacer las operaciones con nuestro usuario.



Para no perder tiempo, solo coloco un Breackpoint en el “ lea rcx, [rsp+98h+Str1] ; Str1”.

Como hice en el anterior ejercicio, ingreso un nombre (“karasu”) y una contraseña (“12345”) de prueba.

```

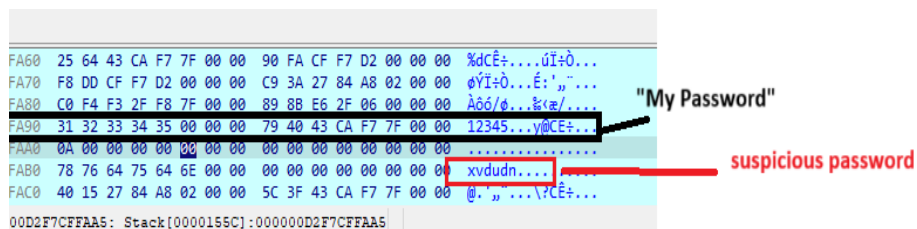
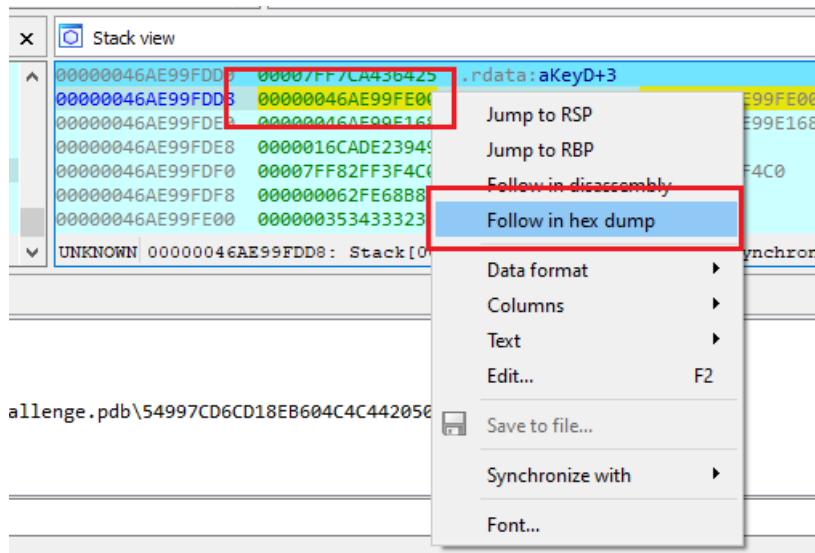
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: karasu
Enter the generated key: 12345

```

Allí el programa se detiene, entonces pensé en revisar en el Stack – Follow in hex dump – en la línea “00000046AE99FDD8 00000046AE99FE00”.



Luego sigue el programa y termina indicando que no es la contraseña adecuada, por lo que, decidí probar al estilo fuerza bruta – entonces, name(“karasu”) y contraseña (“xvdudn”) y es aceptada correctamente.

```

C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: karasu
Enter the generated key: xvdudn
[+] Correct! Generated key: xvdudn

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level:

```

Efectivamente si lo resuelve, pero momento, alguna vez me dijeron: “¿por qué no probas con otro nombre?” Entonces lo probé:

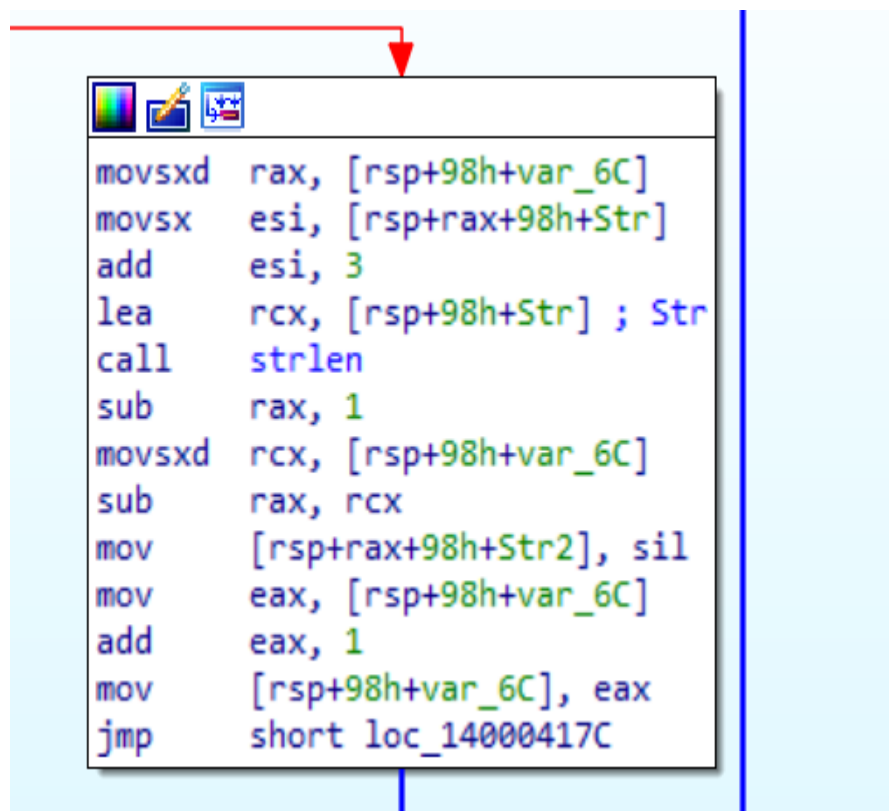
```
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: solid
Enter the generated key: xvdudn
[!] Wrong key, try again

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: _
```

Por supuesto no funcionó.

Luego de analizarlo detenidamente, me di cuenta de que la respuesta estaba ahí a la vista, y era el famoso bucle.



Sin dudas, lo que hace el bucle es tomar la cadena del usuario tecleado previamente e irlo recorriendo y sumarle +3 a cada carácter – luego lo invierte. Ejemplo con karasu →

Carácter de Entrada	Valor ASCII Original	Operación (ASCII+3)	Nuevo Valor ASCII	Carácter Cifrado
k	107	107+3	110	n
a	97	97+3	100	d
r	114	114+3	117	u
a	97	97+3	100	d
s	115	115+3	118	v
u	117	117+3	120	x

Al principio debo admitir, caí en la trampa, por querer apresurar las cosas probé usuario karasu y contraseña ndudvx y lógicamente fracasé. Pero al tomarme unos minutos, me di cuenta de que me faltaba un paso, y era invertirlo, es decir que el programa en sí, toma cada carácter y luego lo invierte.

Realice algunas pruebas por ejemplo:

```

7: Level 2 (obfuscated code)
0. Exit
tables Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: solid
Enter the generated key: glorv
[+] Correct! Generated key: glorv

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)

```

```

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: ricardo
Enter the generated key: rgudflu
[+] Correct! Generated key: rgudflu

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)

```

Por lo tanto, mi teoría era correcta. Y diseñé el script el cual dejo una captura aquí, y lo adjuntare como “Challenge2-BinaryGeckoENG.py”

Nivel 3 (XOR decryption)

Comencemos con el nivel 3:

```
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption) ←
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
10. Exit
Select a level: 3

=== Level 3 ===
Goal: Key is XOR-encrypted
Enter the final key: 123456 ← My Test Password
V
===== Reverse Engineering Challenge =====
```

Bueno, el primer problema que tuve es que, le brindaba cualquier contraseña, pero no me decía más nada, así abrí el IDA y comencé a intentar analizar la situación.

```
sub_140004250 proc near
var_50= dword ptr -50h
var_4C= dword ptr -4Ch
Str2= byte ptr -48h
var_44= byte ptr -44h
Str= byte ptr -28h
var_8= qword ptr -8

sub     rsp, 78h
mov     rax, cs:__security_cookie
xor     rax, rsp
mov     [rsp+78h+var_8], rax
mov     eax, cs:dword_140006160
mov     [rsp+78h+var_4C], eax
lea     rcx, aLevel3 ; "\n=== Level 3 ===\n"
call    sub_140001250
lea     rcx, aGoalKeyIsXorEn ; "Goal: Key is XOR-encrypted\n"
call    sub_140001250
lea     rcx, aEnterTheFinalK ; "Enter the final key: "
call    sub_140001250
lea     rdx, [rsp+78h+Str]
lea     rcx, a19s ; "%19s"
call    sub_140004020
call    sub_140003F50
lea     rcx, [rsp+78h+Str] ; Str
call    strlen
cmp     rax, 4
jbe     short loc_1400042B9
```

Buy length with 4 → The length of the entered string (key) is calculated and saved in rax

Como recién comienzo con esto de ingeniería inversa hay cosas que tengo que ver y rever varias veces. Y en una de las pruebas, salió apareció y viendo la pantalla anterior, claro, mi primer test password no funcionó porque yo probe con 6 caracteres (123456) y esta vez probe con 4 al azar (z{xy):

```

rt      === Level 3 ===
xit_tab Goal: Key is XOR-encrypted
        Enter the final key: z{xy
        real key: X0R3
tion    [-] Wrong. Keep trying
gth(voi

```

¿Que la clave XOR3 es la real? No existe nadie tan bueno en este mundo para que te indiquen cual es la clave correcta, perdonen si dudo de estas intenciones, aun así, decidí probarlo.

Y en efecto, en teoría es esa clave:

```

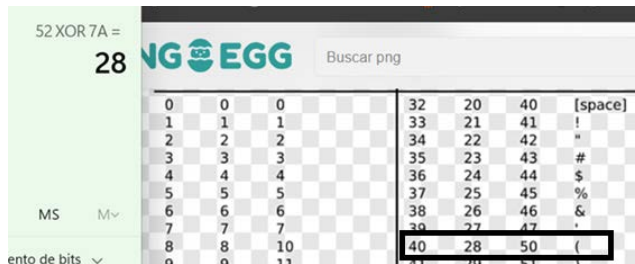
t_tab   === Level 3 ===
on      Goal: Key is XOR-encrypted
h(voi   Enter the final key: X0R3
de      real key: X0R3
        [+] Congratulations! Decrypted key: X0R3
        ===== Reverse Engineering Challenge =====

```

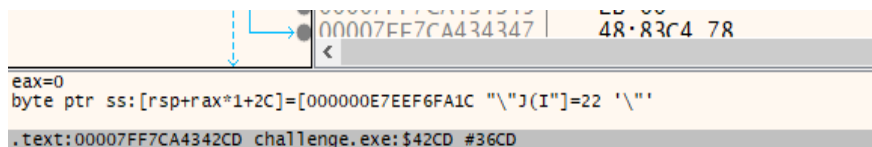
Pero al ser un desafío de BinaryGecko y ya estando a nivel 3 de dificultad, no me lo creo que sea tan fácil, así que seguí investigando un poco más. Utilizaré x64 para revisar de forma dinámica.

B5	76 02	jbe	challenge.7FF7CA4342B9	Validate that the password is 4
B7	EB 7B	jmp	challenge.7FF7CA434334	
B9	C74424 28 00000000	mov	dword ptr ss:[rsp+28],0	
C1	837C24 28 04	cmp	dword ptr ss:[rsp+28],4	
C6	7D 23	jge	challenge.7FF7CA4342EB	
C8	48:634424 28	movsxd	rax,dword ptr ss:[rsp+28]	
CD	0FBE4404 2C	movsx	eax,byte ptr ss:[rsp+rax+2C]	XOR with 7A
D2	83F0 7A	xor	eax,7A	
D5	48:634C24 28	movsxd	rcx,dword ptr ss:[rsp+28]	
DA	88440C 30	mov	byte ptr ss:[rsp+rcx+30],al	
DE	8B4424 28	mov	eax,dword ptr ss:[rsp+28]	
E2	83C0 01	add	eax,1	
E5	894424 28	mov	dword ptr ss:[rsp+28],eax	
E9	EB D6	jmp	challenge.7FF7CA4342C1	

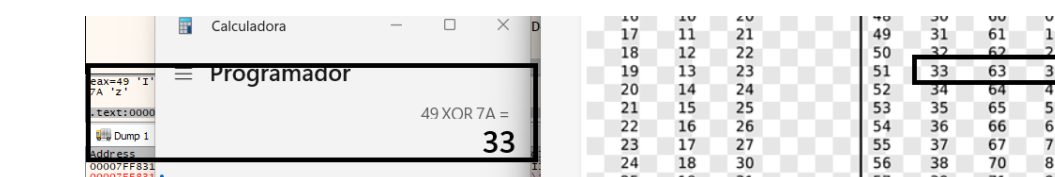
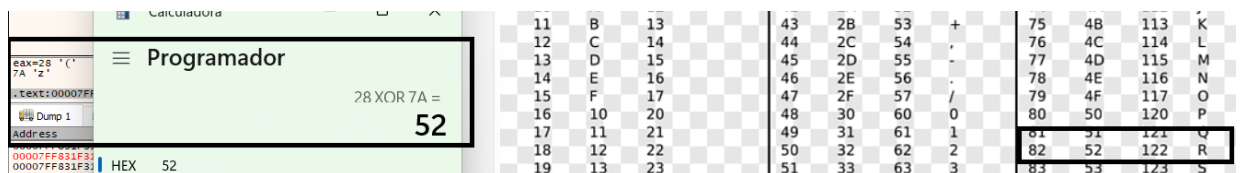
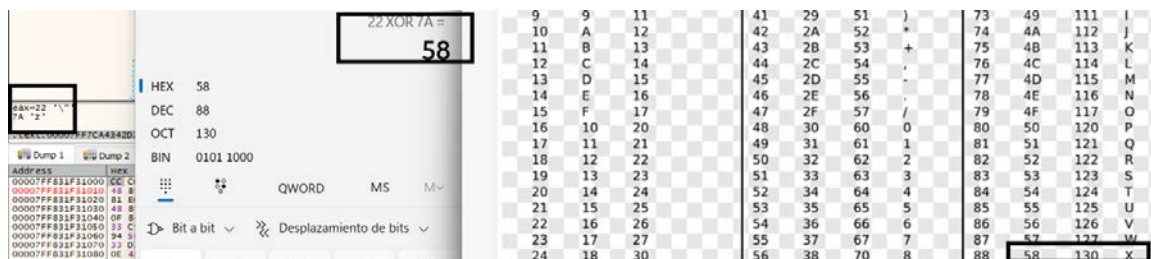
Lo que hace en si el programa es que realiza la operación XOR EAX,7A entonces esto quiere decir que realizara lo siguiente:



Adicionalmente para justificarlo, el x64 indica:



Si seguimos lo indicado:



Por lo que luego:

Assembly code snippet:

```

00007FF7CA4342E5 894424 28 mov dword ptr ss:[rsp+28],eax
00007FF7CA4342E9 EB D6 jmp challenge.7FF7CA4342C1
00007FF7CA4342EB C64424 34 00 mov byte ptr ss:[rsp+34],0
00007FF7CA4342F0 48:8D5424 30 lea rdx,qword ptr ss:[rsp+30]
00007FF7CA4342F5 48:8D0D 74230000 lea rcx,qword ptr ds:[7FF7CA436670]

```

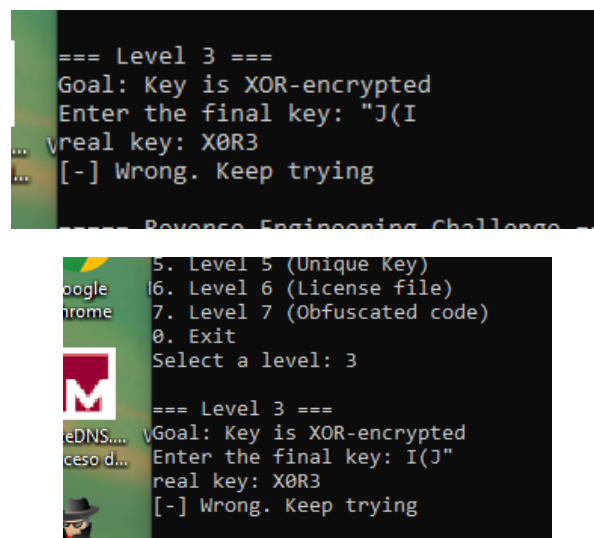
Memory dump snippet:

Address	Hex	ASCII
000000E7EEF6F9F0	C0 F4 F3 2F E8 7F 00 00	A06/0...@u61c...
000000E7EEF6FA00	78 00 00 00 00 00 00 00	X...00000000
000000E7EEF6FA10	0A 00 00 00 00 00 00 00	04 00 00 00 22 4A 28 49
000000E7EEF6FA20	58 30 52 33 00 00 00 00	00 00 00 00 00 00 00 00
000000E7EEF6FA30	00 1F 39 84 6C 02 00 00	SC 3F 43 CA F7 7F 00 00
000000E7EEF6FA40	31 39 34 35 00 7F 00 00	30 F4 F3 2F E8 7F 00 00
000000E7EEF6FA50	90 F4 F3 2F E8 7F 00 00	60 BF 32 84 6C 02 00 00

Labels in the image:

- "decrypted" key
- My Key
- correct key

Por lo que el resultado debería ser **"J(I** – o si fuera inverso – **I(J"**



En resumen, evidentemente la única clave aceptada es XOR3 –

Nivel 4 (Math Operations)

```

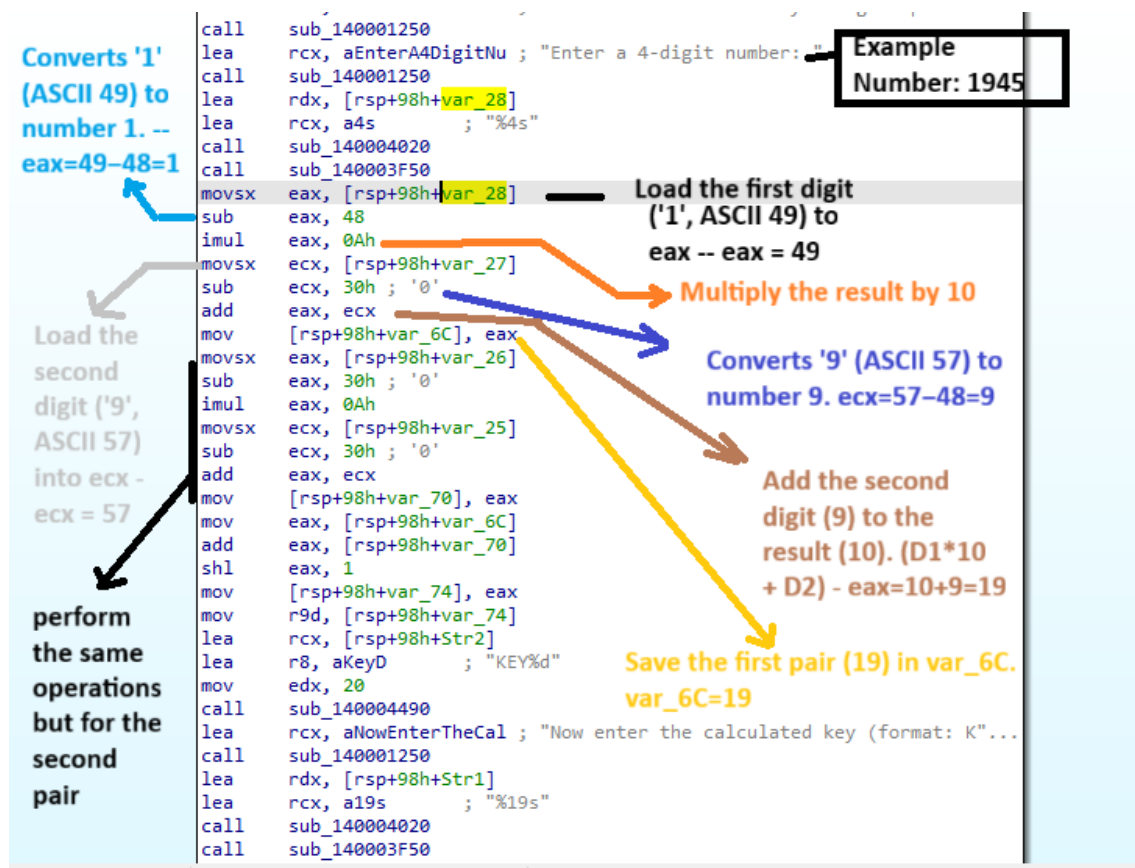
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
10. Exit
Select a level: 4

=== Level 4 ===
Goal: Calculate the key using simple math operations
Enter a 4-digit number: 1234
Now enter the calculated key (format: KEYXXXX): KEY1234
[!] Wrong key, try again
  
```

2 Test Password (Format KEYNNNN)

Bien, comencemos con el cuarto desafío. Bueno primero pide que el usuario ingrese 4 números – Luego vuelve a pedir la clave calculada con formato “KEYXXXX” -

Luego de analizarlo estáticamente y hacer algunas pruebas de forma dinámica, me di cuenta que no era tan problemático como parecía, lo que ocurre es que hace varias operaciones y eso ocasiona que a veces uno se pierda.

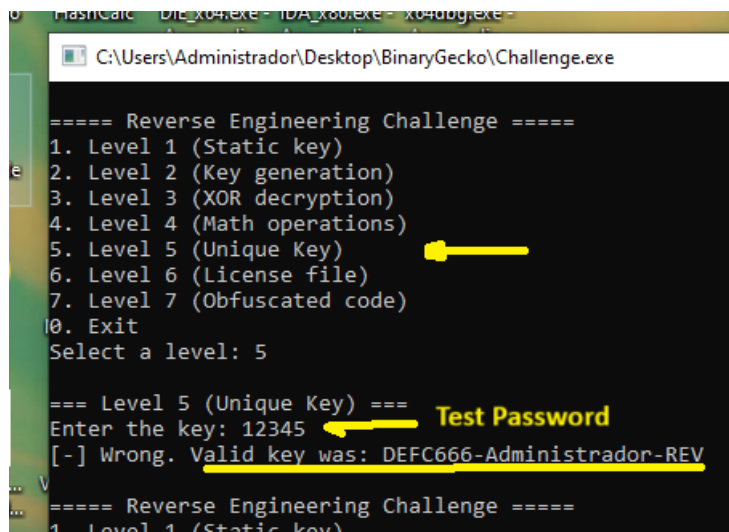


Pido disculpas de ante mano con tantos colores, pero no se me ocurrió otra forma de intentar explicarlo de una mejor forma.

Pero en resumidas cuentas lo que realiza el programa es – Solicita un número de 4 dígitos (en nuestro ejemplo será 1945) – luego separa en dos – es decir toma 19 por un lado – y 45 por el otro – luego los suma $\rightarrow 19+45 = 64$ – luego lo multiplica por 2 – $64 \times 2 = 128$ y luego le concatena la palabra KEY – Lógicamente, luego compara “KEY128” con 2da clave que se ingresa y si no son iguales, salta a “chico malo” y finaliza. Si no, salta a “chico bueno” y resolvimos el ejercicio.

Aunque no lo soliciten expresamente, me tome la libertad de realizar un pequeño keygen – Adjuntare dicho script con el nombre “Challenge4-BinaryGeckoENG.py” referido a este punto.

Nivel 5 (Unique Key)



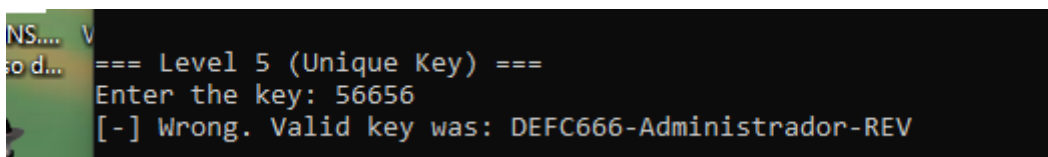
```
==== Reverse Engineering Challenge ====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 5

=== Level 5 (Unique Key) ===
Enter the key: 12345
[-] Wrong. Valid key was: DEFC666-Administrador-REV

==== Reverse Engineering Challenge ====
1. Level 1 (Static key)
```

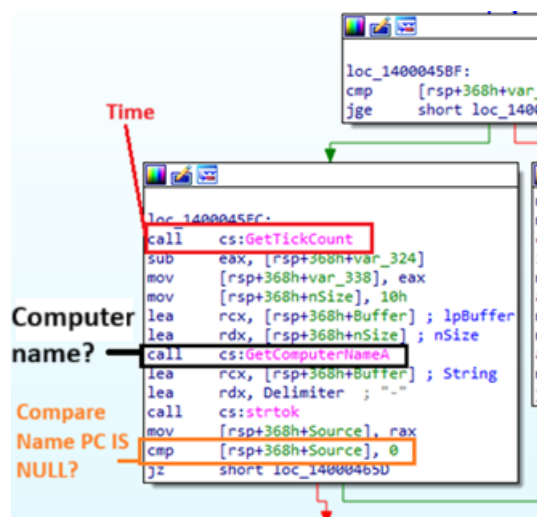
Bueno comenzando el desafío numero 5 me solicitan una clave (no determina formato ni longitud. Para mi sorpresa, no solo informa que no es esa la clave y que la clave valida es “DEFC666-Administrador-REV”.

Intente con otra clave totalmente distinta, pero informa exactamente lo mismo:



```
==== Level 5 (Unique Key) ===
Enter the key: 56656
[-] Wrong. Valid key was: DEFC666-Administrador-REV
```

Al iniciarlo en IDA PRO – y hacer un chequeo rápido, me doy cuenta de lo siguiente:

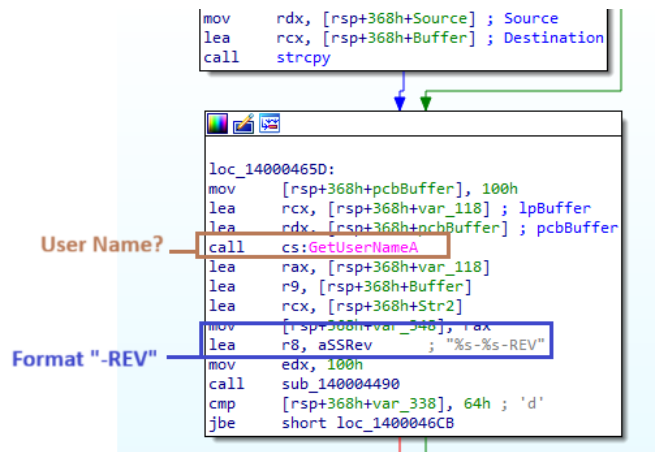


Bueno, parece que aquí tenemos algunos detalles que me gustaría expresar.

Según MSDN “GetTickCount”: Recupera la cantidad de milisegundos que han transcurrido desde que se inició el sistema, hasta 49,7 días.

Según MSDN “GetComputerNameA”: Recupera el nombre NetBIOS del equipo local. Este nombre se establece al iniciar el sistema, cuando el sistema lo lee del Registro.

Ademas compara/comprueba (al menos es una estimación mía por cómo viene dado [RSP+368h+Source],0 – si se encontró un token (si el nombre del equipo no estaba vacío).

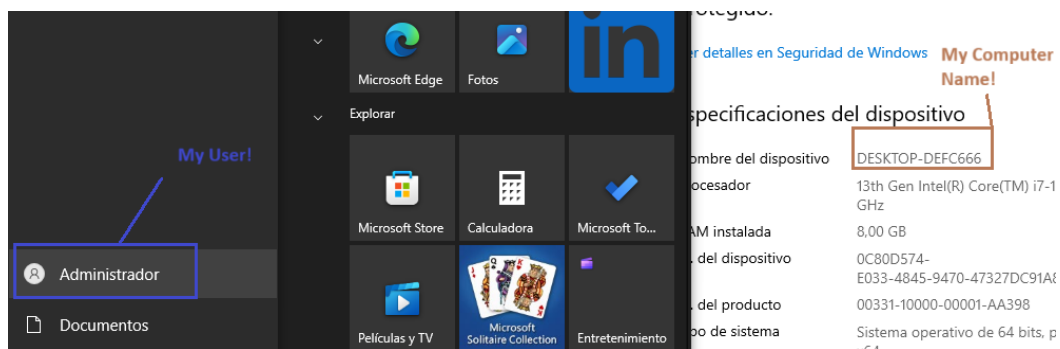


En la imagen anterior, nos muestra dos cosas importantes:

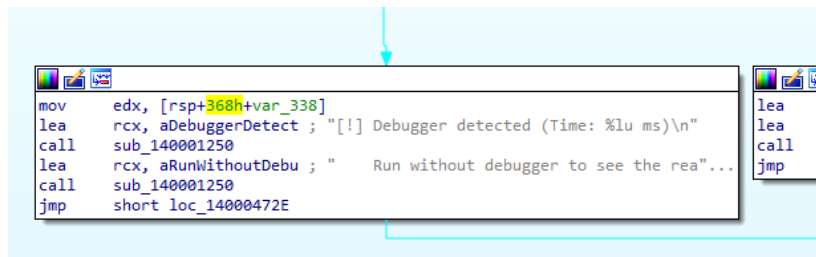
Según MSDN “GetUserNameA”: Recupera el nombre del usuario asociado al subproceso actual.

Además, más abajo prepara para tener un formato -REV

Si lo pensamos un minuto tiene algo de sentido si lo comparamos con lo que nos mencionaba el programa cuando hicimos el intento – nos decía que la clave era “DEFC666-Administrador-REV” – Entonces tiene sentido, ya que toma el nombre de la PC + El nombre del usuario y le concatena “-REV”
En mi entorno:



Por supuesto, esto tiene adicionalmente “Chico Bueno” y “Chico Malo” pero además, tiene un bloque de código para Antidebugger que tiene relación con el tiempo.

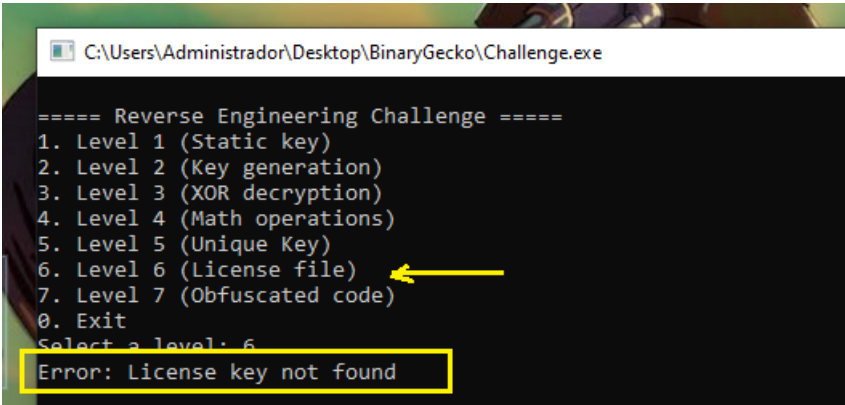


Ahora bien, como también necesito practicar algo de programación, se me ocurrió hacer otro Script (en si no es necesario, pero de todos modos lo haré).

Se llamará Challenge5-BinaryGeckoENG.py y será enviado junto con este documento y los otros scripts.

Nivel 6 (Licence file)

Iniciemos el ante ultimo desafío vamos primero a ejecutarlo



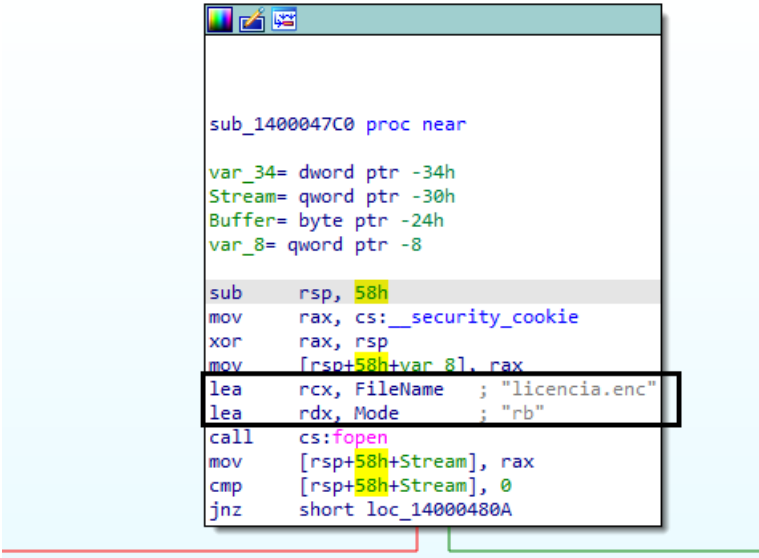
```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 6
Error: License key not found
```

Ok, entonces ni bien lo inicio, indica “Error: License key not found”

Esto quiere decir que espera un archivo, según mi experiencia.

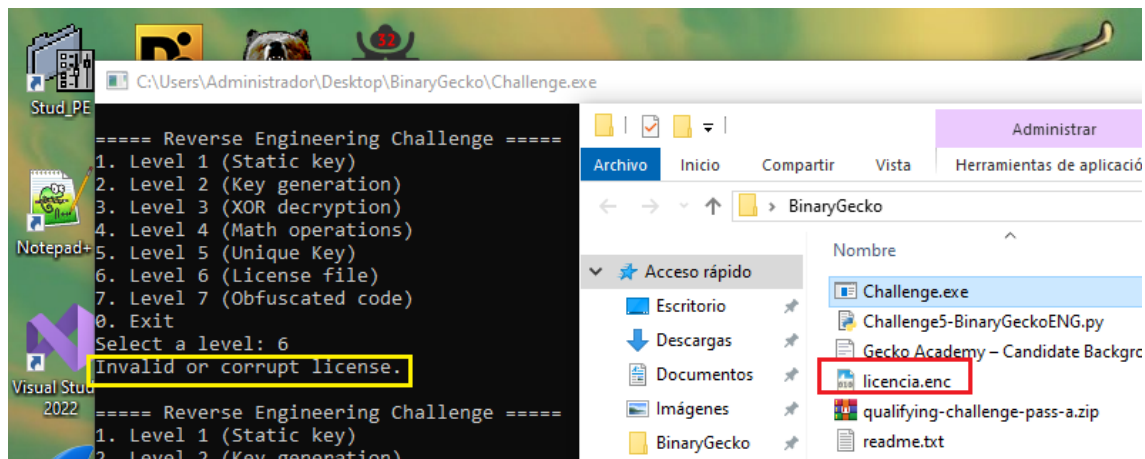
Al abrir en el IDA Pro – esta muy claro lo que mencionaba antes



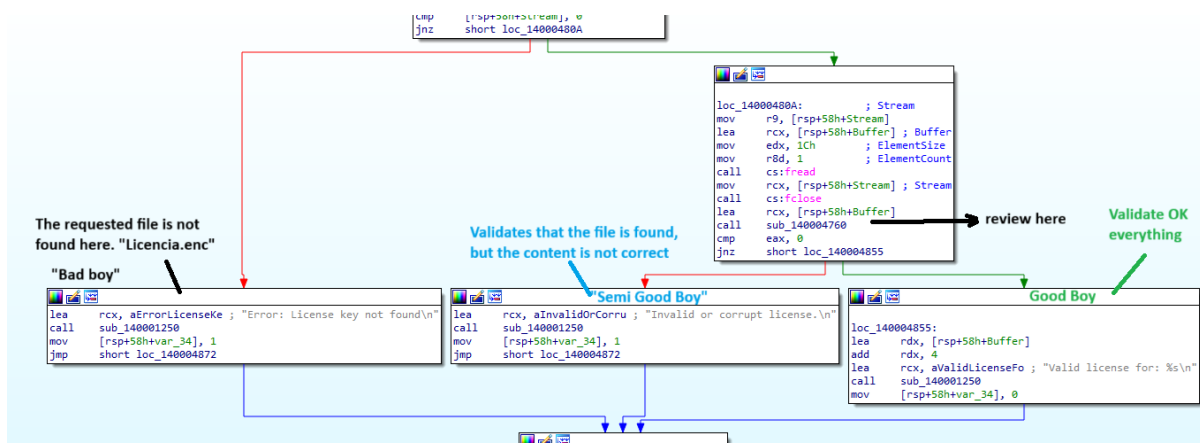
```
sub_1400047C0 proc near
var_34= dword ptr -34h
Stream= qword ptr -30h
Buffer= byte ptr -24h
var_8= qword ptr -8

sub     rsp, 58h
mov     rax, cs:_security_cookie
xor     rax, rsp
mov     [rsp+58h+var_8], rax
lea     rcx, FileName      ; "licencia.enc"
lea     rdx, Mode          ; "rb"
call    cs:fopen
mov     [rsp+58h+Stream], rax
cmp     [rsp+58h+Stream], 0
jnz     short loc_14000480A
```

No solo que espera un archivo “licencia.enc” sino que además tiene permiso para mínimamente leerlo



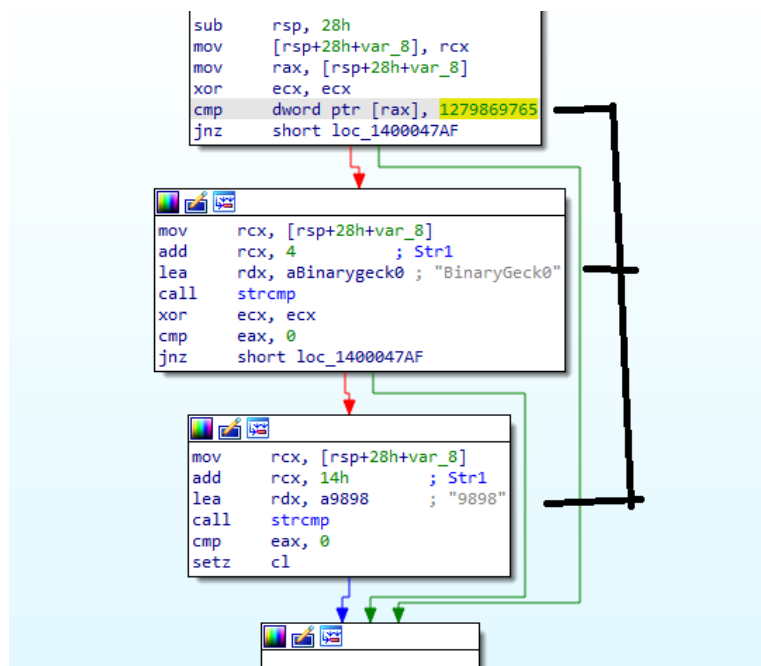
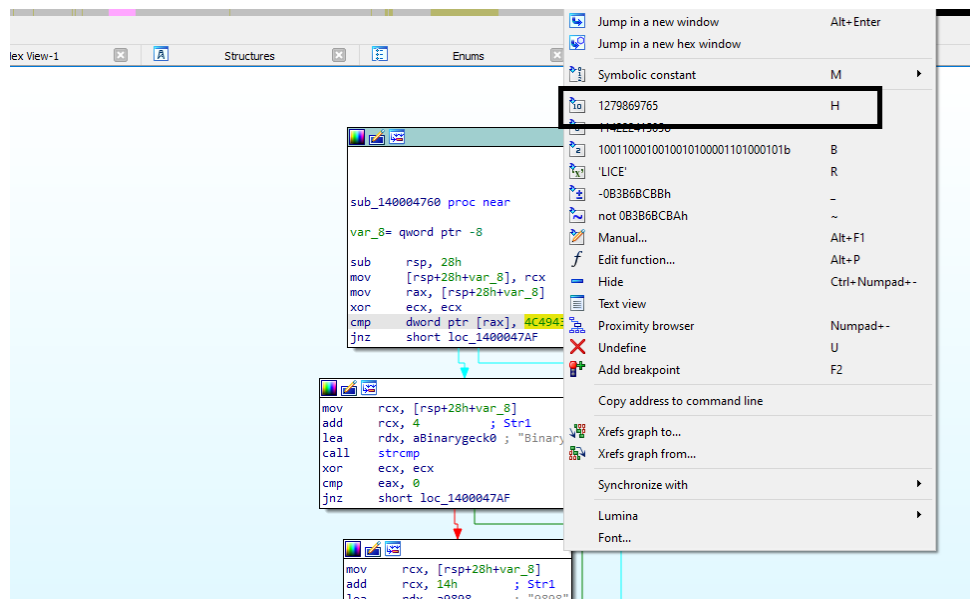
Para validar mi idea anterior, se me ocurrió, crear un archivo llamado “licencia.enc” – en realidad es un block de notas con ese nombre y le cambie la extensión. Ahora bien, vemos que el mensaje cambio, confirmando así las dos cosas: La primera que necesita dicho archivo y la segunda, que evalúa el contenido del mismo.



En esta imagen, por si no se ve clara, lo siento pensé que era más pequeño – tenemos una función a investigar, luego 3 caminos a los cuales defini como “chico malo” que es cuando no existe el archivo licencia.enc – Luego está “chico semi bueno” – el cual valida que este el archivo pero también que el contenido sea el correcto – y “chico bueno” el cual valida que esta OK el archivo y su contenido.

Luego en la funcion pendiente para revisar encuentro que compara con la constante “1279869765” y no es un dato menor, ya que en una secuencia de caracteres ASCII corresponde a la cadena “Lice”

Esto es así porque: 1279869765 en Big Endian → 4C,49,43,45 LICE // en Little Endian sería ECIL (45 43 49 4C)

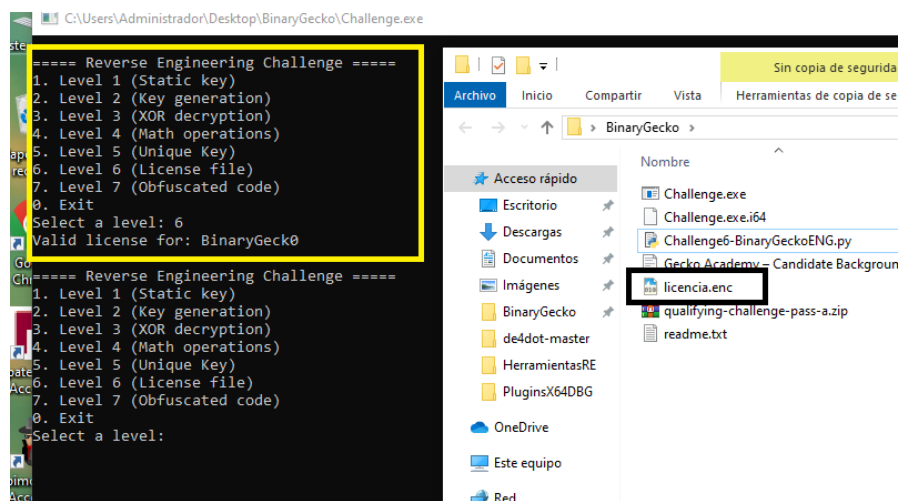


Y luego vemos que está la cadena BinaryGeck0 y 9898 – así que, reuniendo toda esta información, se me ocurrió que el archivo podría contener las siguientes strings “LICE” + “BinaryGeck0” + “9898” – voy a ser honesto en este punto también, la verdad probe varias formas. Y siempre guardaba y probaba distintas combinaciones y nunca daba y no entendía por qué, hasta que recordé un ejercicio parecido, el cual en vez de escribirlo yo de forma manual en un documento como un txt – había que escribirlo lo mismo en bytes.

Es decir, por ejemplo “ECIL+BinaryGeck0 +9898” – así que volví a realizar pruebas y seguía fracasando, hasta que me di cuenta de que era como lo estaba escribiendo en el archivo. Finalmente me di cuenta de que debería escribirlo así (Estoy abriendo el archivo Licence.enc con Notepad++ y usando el plugin “HEX Editor”

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	Dump
00	45	43	49	4c	42	69	6e	61	72	79	47	65	63	6b	30	ECILBinaryGeck0.
10	00	00	00	00	39	38	39	38	00							9898.

Necesitaba completar la secuencia con 0 o alguna otra cosa para que me tomara bien la clave (esto sinceramente me gustaría en algún momento sacarme la duda por que muy bien no lo entendí la causa – imagino que s por que necesita completar la cantidad de bytes).

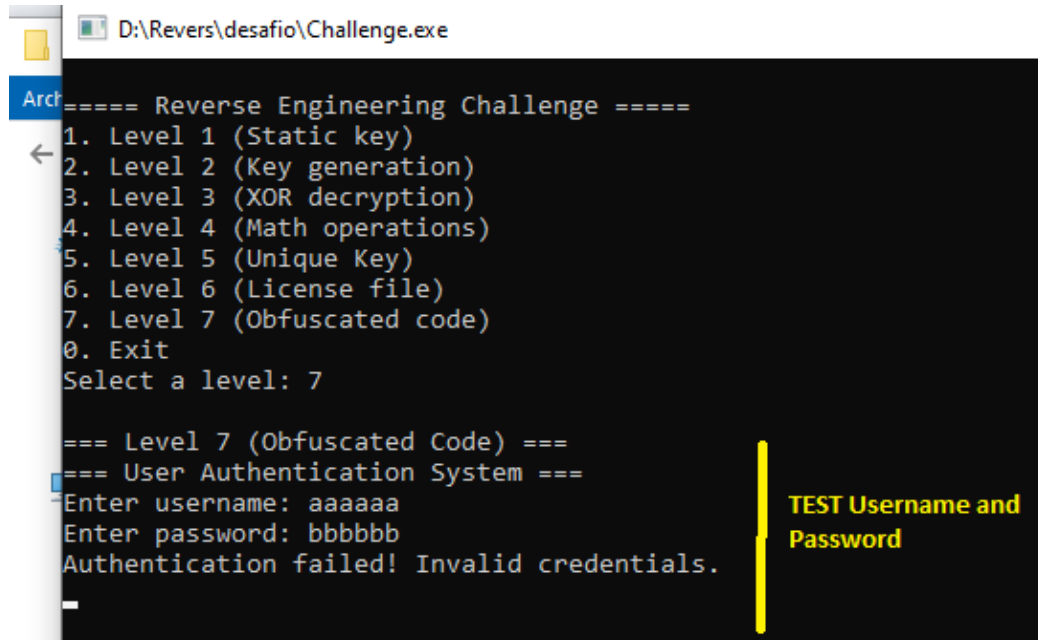


Así, podemos decir que finalmente di con la solución, creando el archivo “Licencia.enc” con el contenido ya visto, al ejecutarlo lo valida OK –

Por supuesto que genere un script (el cual como los anteriores, también será adjuntado) con el nombre “Challenge6-BinaryGeckoENG.py”

Nivel 7 (Obfuscated code)

Al iniciar el ultimo desafío, directamente me pide un usuario y una contraseña, al no ser correcta, nos dice "Authentication failed! Invalid credentials. Y luego finaliza el ejecutable.

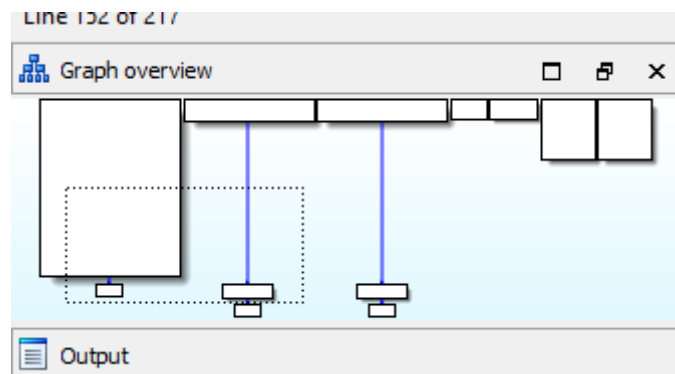


```
D:\Revers\desafio\Challenge.exe

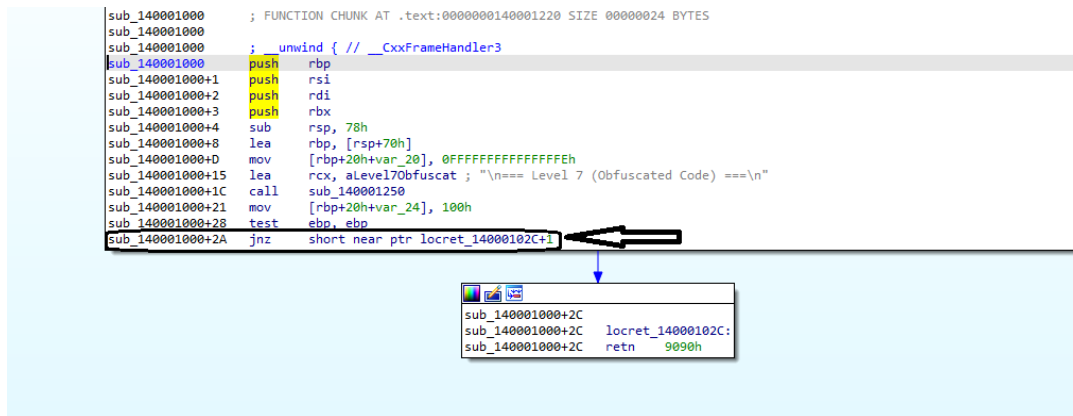
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 7

=== Level 7 (Obfuscated Code) ===
=== User Authentication System ===
Enter username: aaaaaa
Enter password: bbbbbb
Authentication failed! Invalid credentials.
```

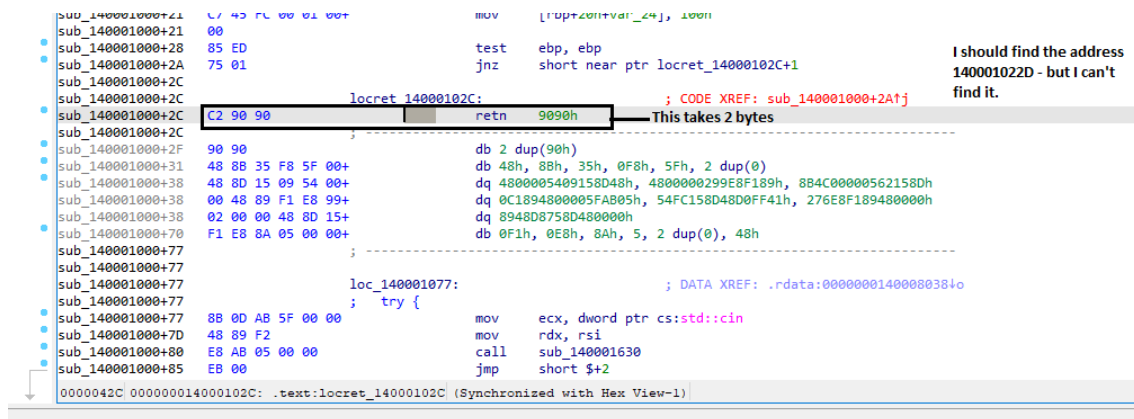
Efectivamente el código está Ofuscado, nos podremos dar cuenta por algunas señales como esta:



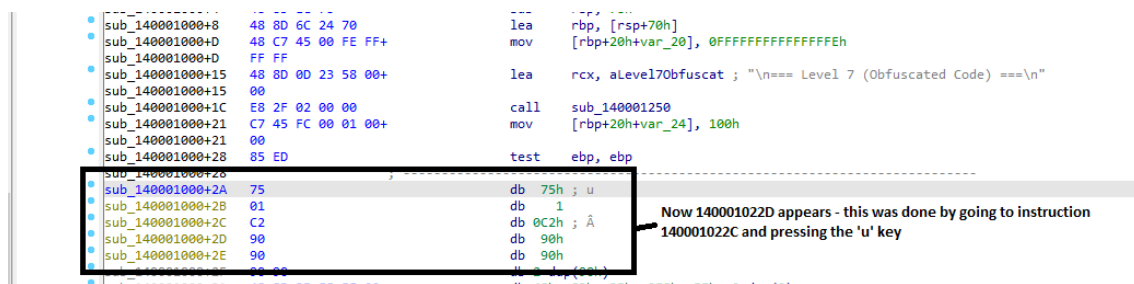
Como prueba, abro IDA Pro y veo líneas como la marcada – que indican +”n” – en este caso +1 – eso es porque no identifica correctamente.

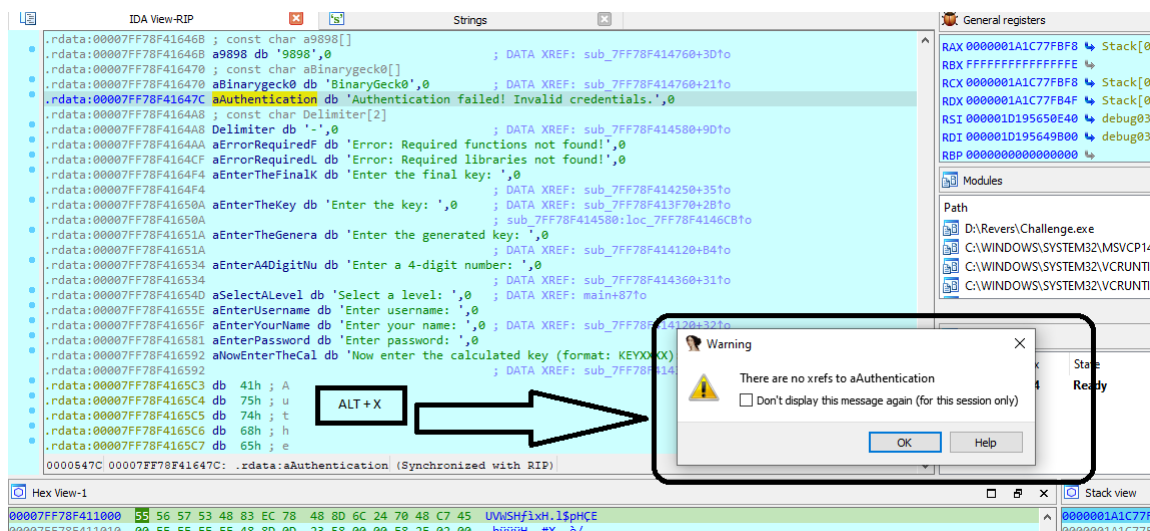


Aquí indica claramente que hay un C2 90 90 y en teoría debería ir a la dirección 140001022D – pero yo aquí no la encuentro.



Luego, podríamos ir recorriendo el extenso código e ir reconstruyendo todo lo que se permita.



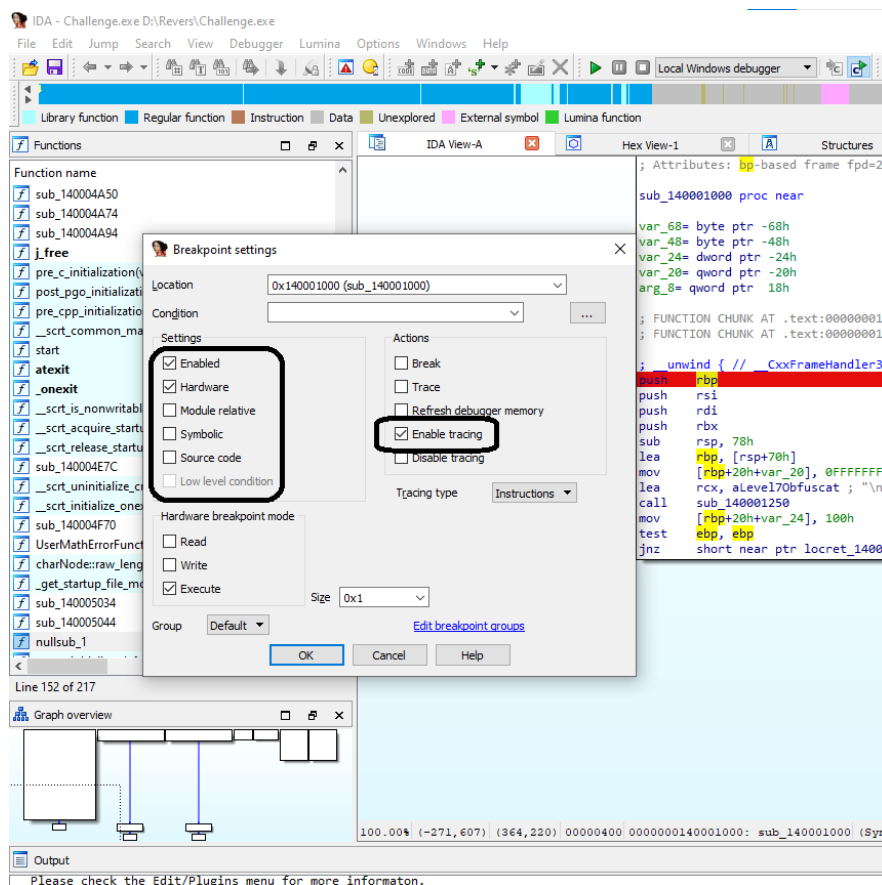


Aquí le das Alt + X por ejemplo para ver la función de la string “Authentication failed!...” y da ese error.

Pero no tengo mucho tiempo y la verdad, que alguien con mayor capacidad, podría generar un script si existiera un patrón e ir quitando la parte basura y reconstruyendo así el código, pero no es mi caso.

Primero voy a poner un Hardware breakpoint – y habilito “Enable Tracing” -

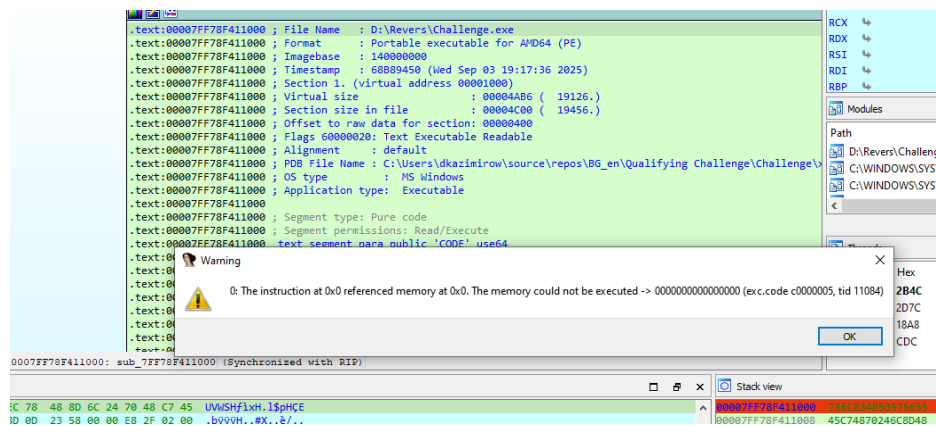
Con esto, podre “reconstruirlo” de forma temporal, y darme cuenta como arma el password:



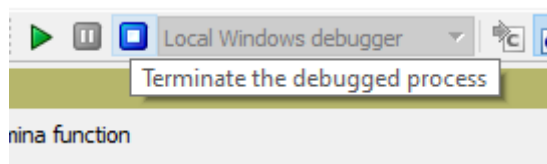
```

.text:00007FF78F411000 ; File Name : D:\Revers\Challenge.exe
.text:00007FF78F411000 ; Format : Portable executable for AMD64 (PE)
.text:00007FF78F411000 ; Imagebase : 140000000
.text:00007FF78F411000 ; Timestamp : 6889450 (Wed Sep 03 19:17:36 2025)
.text:00007FF78F411000 ; Section 1. (virtual address 00001000)
.text:00007FF78F411000 ; Virtual size : 00004AB6 ( 19126.)
.text:00007FF78F411000 ; Section size in file : 00004C00 ( 19456.)
.text:00007FF78F411000 ; Offset to raw data for section: 00000400
.text:00007FF78F411000 ; Flags 60000020: Text Executable Readable
.text:00007FF78F411000 ; Alignment : default
.text:00007FF78F411000 ; PDB File Name : C:\Users\dkazimirow\source\repos\BG_en\Qualifying Challenge\Challenge\x64\Release\Challenge.pdb
.text:00007FF78F411000 ; OS type : MS Windows
.text:00007FF78F411000 ; Application type: Executable
.text:00007FF78F411000 ; Segment type: Pure code
.text:00007FF78F411000 ; Segment permissions: Read/Execute
.text:00007FF78F411000 ; Text segment para public 'CODE' use64
.text:00007FF78F411000 ; Assume cs: text
.text:00007FF78F411000 ; org 7FF78F411000h
.text:00007FF78F411000 ; assume esi:tdll.dll, ds:data, fs:nothing, gs:nothing
.text:00007FF78F411000
.text:00007FF78F411000 ; Attributes: bp-based frame fpu=20h
.text:00007FF78F411000
.text:00007FF78F411000 sub_7FF78F411000 proc near
.text:00007FF78F411000
.text:00007FF78F411000 var_68= byte ptr -68h
.text:00007FF78F411000 var_48= byte ptr -48h
.text:00007FF78F411000 var_24= dword ptr -24h
.text:00007FF78F411000 var_20= qword ptr -20h
.text:00007FF78F411000 arg_8= qword ptr 18h
.text:00007FF78F411000
.text:00007FF78F411000 ; FUNCTION CHUNK AT .text:00007FF78F4111F0 SIZE 00000024 BYTES
.text:00007FF78F411000 ; FUNCTION CHUNK AT .text:00007FF78F411220 SIZE 00000024 BYTES
.text:00007FF78F411000
.text:00007FF78F411000 ; unwind { // CxxFrameHandler3
4,-84} (93,45) 00000400 00007FF78F411000: sub_7FF78F411000 (Synchronized with RIP)

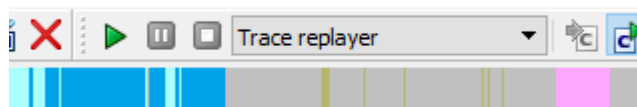
```



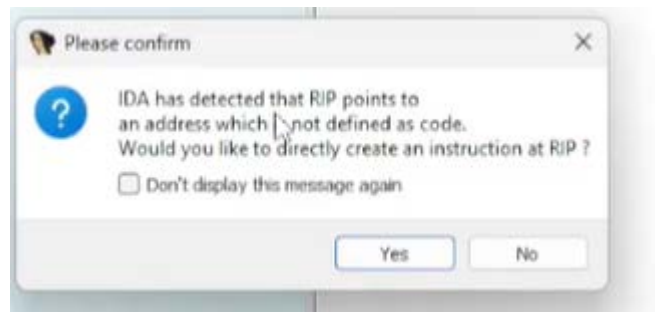
Nos avisa que finalizó.



Cuando llegamos a este punto, detenemos el Debugger y luego seleccionamos "Trace replayer".

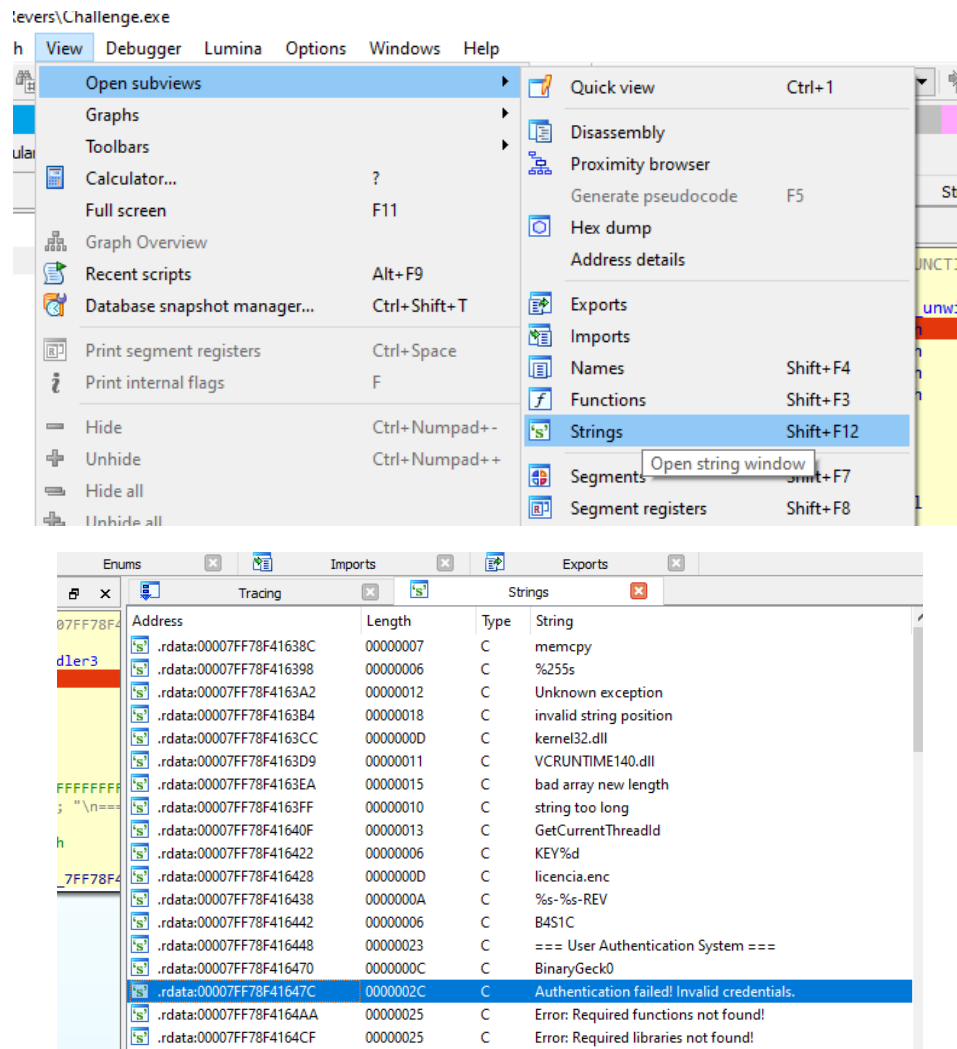


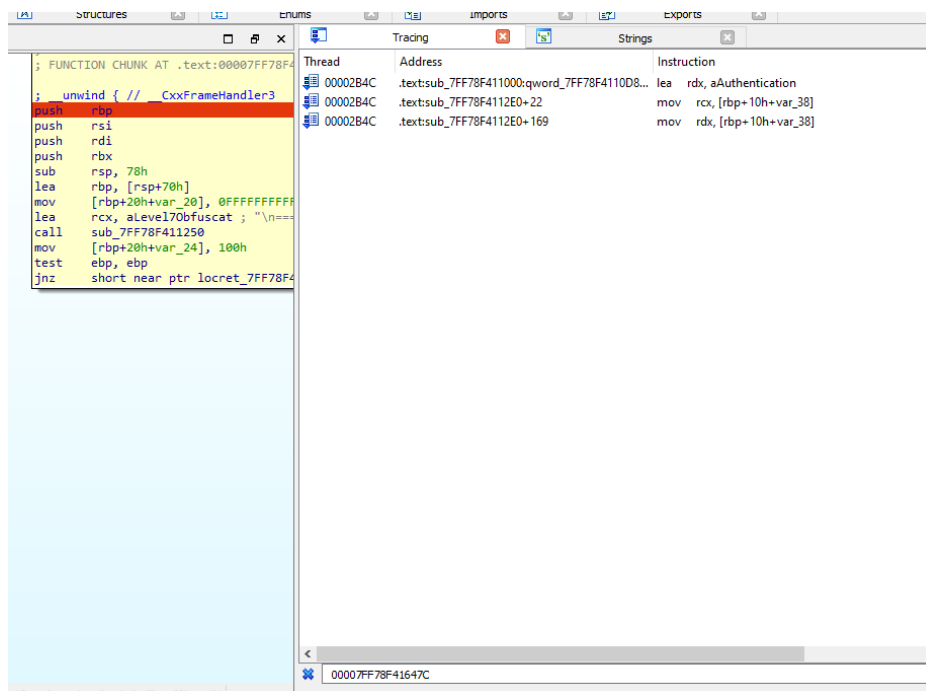
Seguramente al intentar ver alguna función o algo que está dañado aparecerá este mensaje:



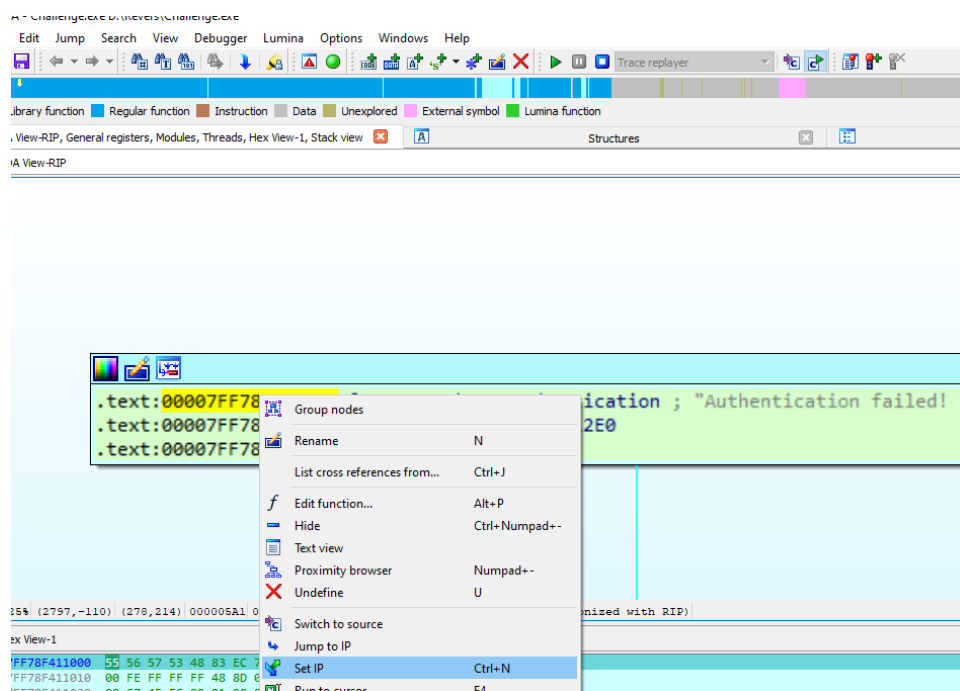
Yo para que no molestar le tilde para que no lo vuelva a mostrar y le puse "Yes".

Luego buscamos la string:

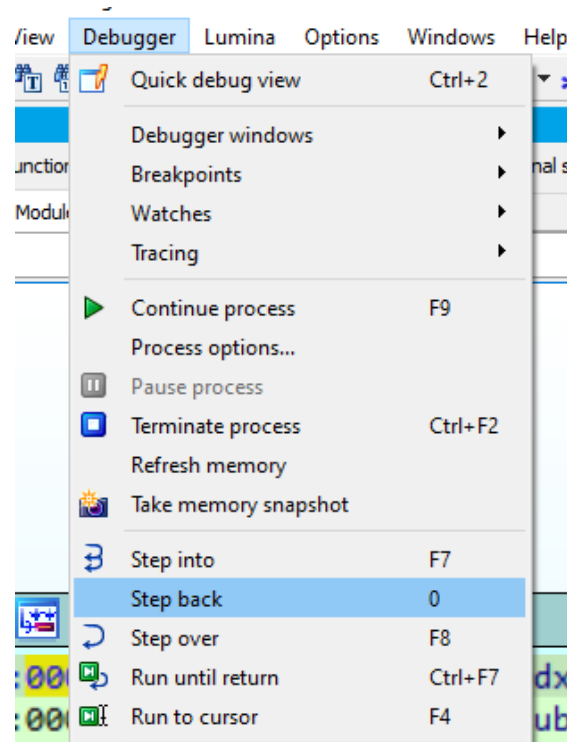




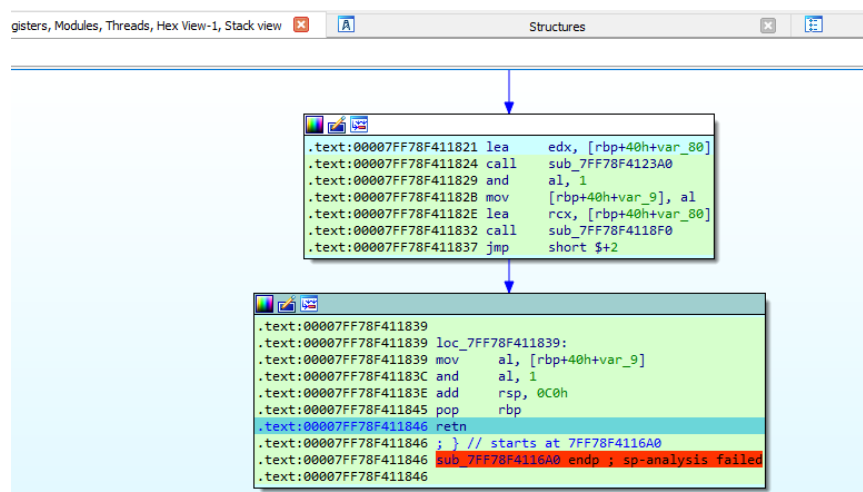
Vemos que su dirección es 00007FF6529F647C – por lo que lo buscamos en la ventana de “Tracing”.



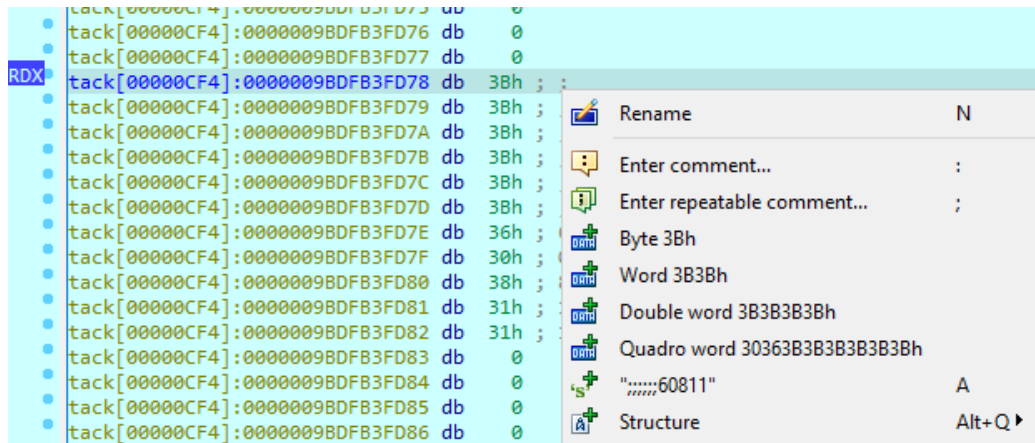
Recordar tomar nota de la dirección donde esta – en este caso era: 00007FF78F4111A1 (varía) – y ahí damos click derecho -> Set IP -



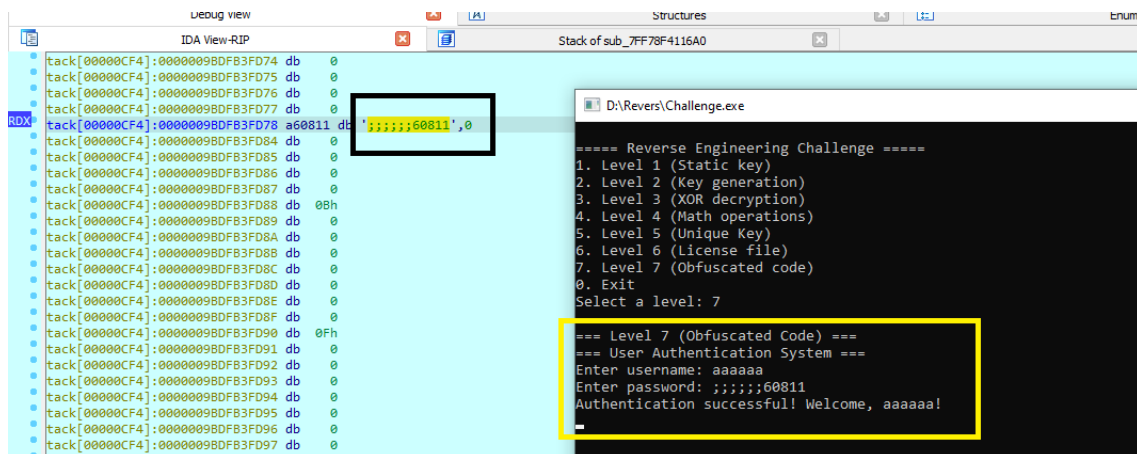
Luego con Step back (en mi caso con el num 0) vamos a reconstruir la sección del código que nos interesa.



Luego colocamos un hardware breackpoint – importante elegir nuevamente “Local Windows debugger” -

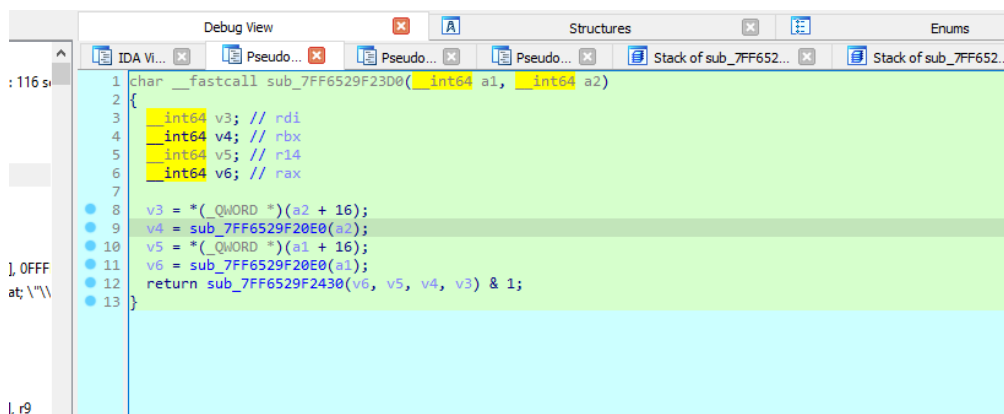


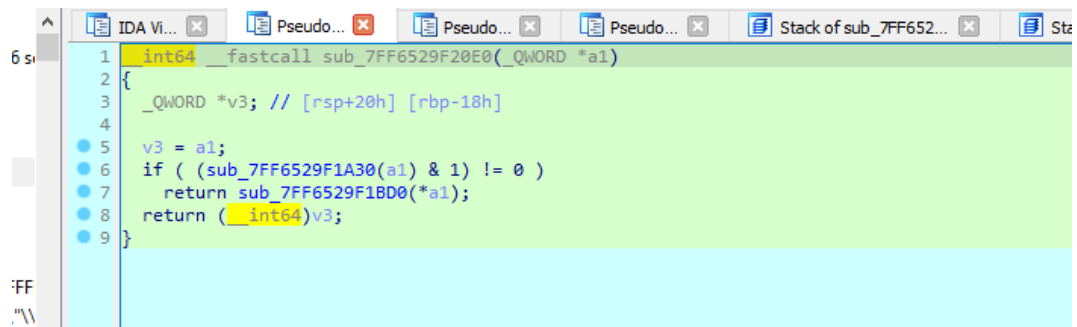
En la línea que dice “var_80” para ver su contenido, le damos click y derecho y seleccionamos la opción correspondiente, o bien tecleamos ‘A’ y arma la string.



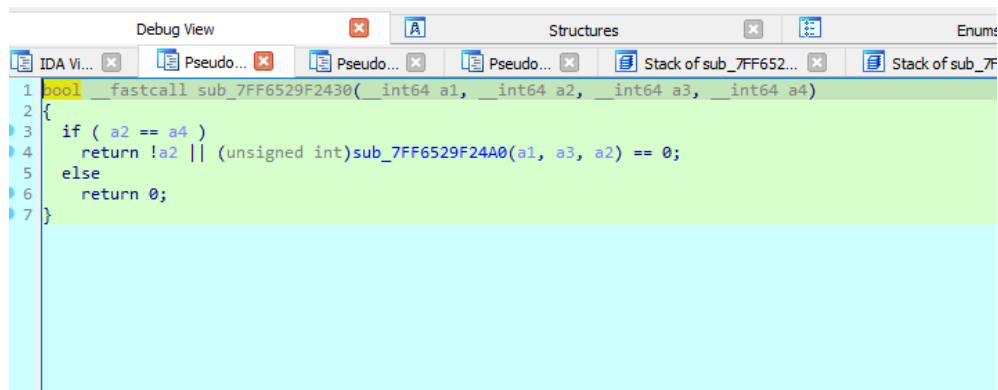
Ahí estaría resuelto. Ahora bien algunos detalles personales que me gustaría compartir:

Mientras iba traceando, la verdad me fue bastante complicado, con F5 – es decir con el pseudocódigo fui encontrando algunas estructuras que me llamaron la atención:

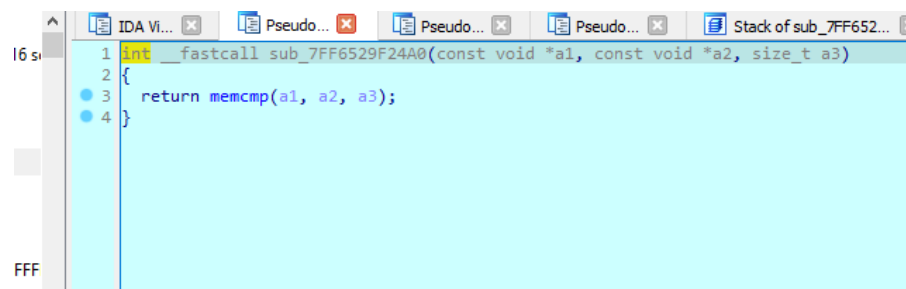




```
1  int64 __fastcall sub_7FF6529F20E0(_QWORD *a1)
2  {
3      _QWORD v3; // [rsp+20h] [rbp-18h]
4
5      v3 = a1;
6      if ( (sub_7FF6529F1A30(a1) & 1) != 0 )
7          return sub_7FF6529F1BD0(*a1);
8      return (int64)v3;
9  }
```



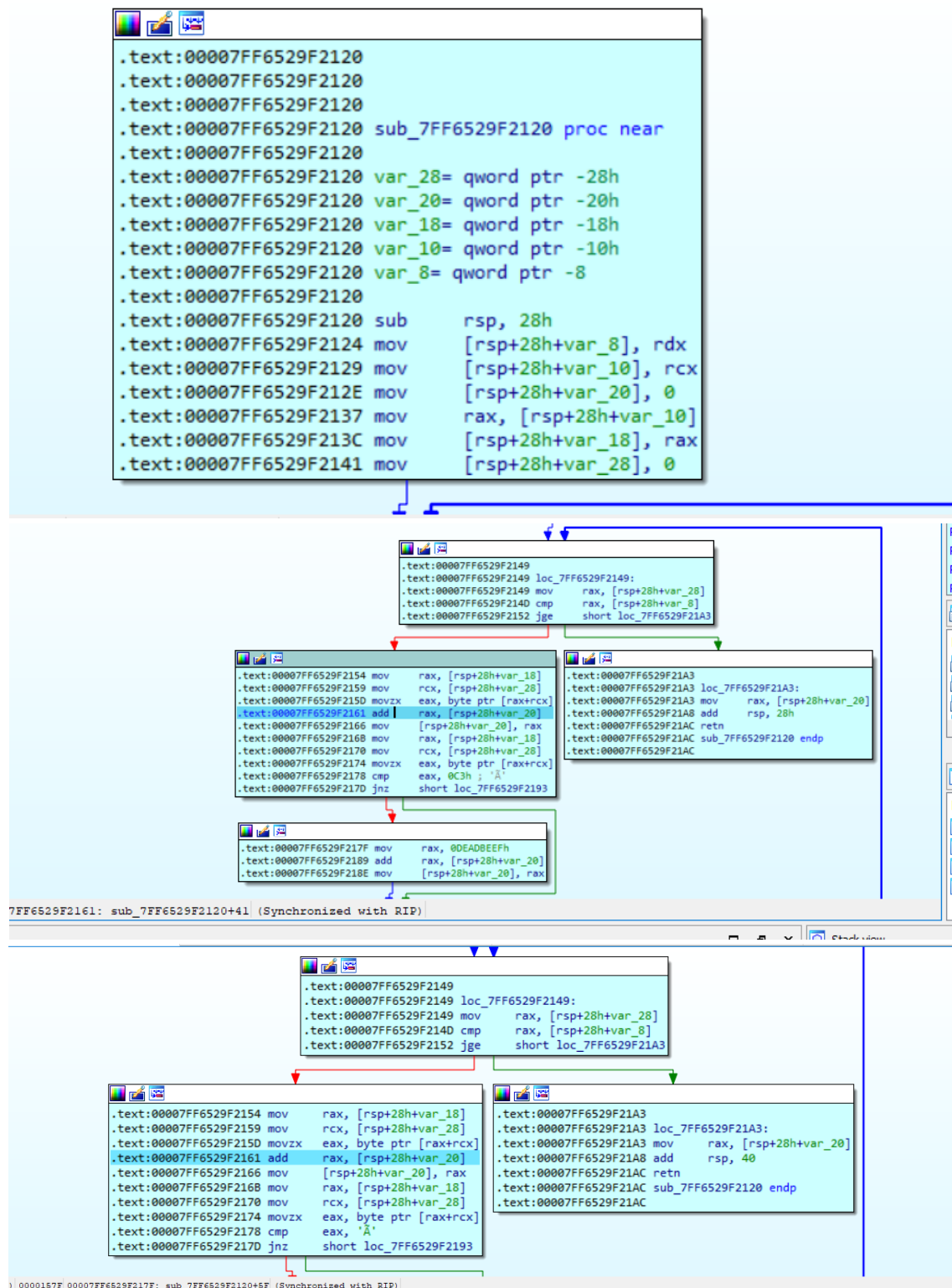
```
1  bool __fastcall sub_7FF6529F2430(_int64 a1, _int64 a2, _int64 a3, _int64 a4)
2  {
3      if ( a2 == a4 )
4          return !a2 || (unsigned int)sub_7FF6529F24A0(a1, a3, a2) == 0;
5      else
6          return 0;
7  }
```

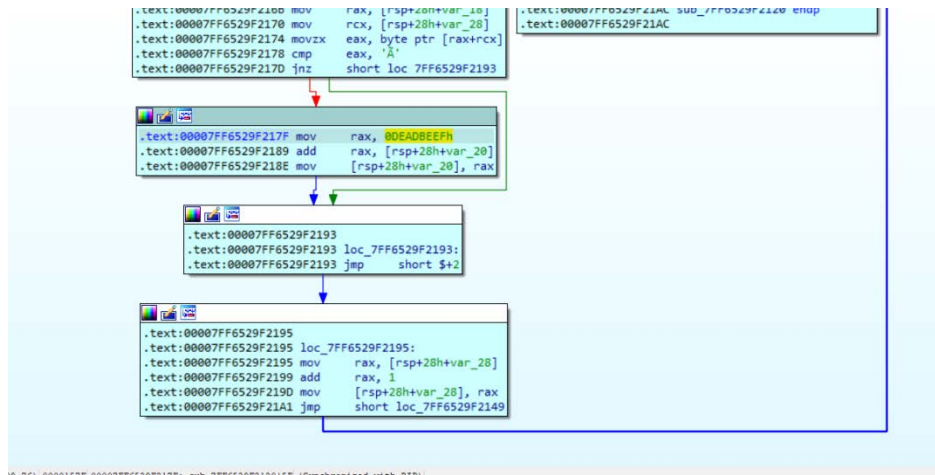


```
1  int __fastcall sub_7FF6529F24A0(const void *a1, const void *a2, size_t a3)
2  {
3      return memcmp(a1, a2, a3);
4  }
```

Todo esto, solo es para marear, lo que importa es que llega a hacer la operación con memcmp.

Luego siguiendo con el rastreo llegue a lo que creo que es el bucle más importante y donde – si bien faltan alguno detalles se realiza la mayor parte de los cálculos:





Por ultimo deajo captura de lo que sería el pseudocódigo según IDA PRO:

```

IDA View-RIP                                     Pseudocode-C
1  __int64 __fastcall sub_7FF6529F2120(__int64 a1, __int64 a2)
2  {
3      __int64 i; // [rsp+0h] [rbp-28h]
4      __int64 v4; // [rsp+8h] [rbp-20h]
5
6      v4 = 0i64;
7      for ( i = 0i64; i < a2; ++i )
8      {
9          v4 += *(unsigned __int8 *) (a1 + i);
10         if ( *(unsigned __int8 *) (a1 + i) == '\xC3' )
11             v4 += 3735928559i64;
12     }
13     return v4;
14 }

```


Conclusión final

Como párrafos finales, no quería perder la oportunidad de agradecerles por los desafíos, la verdad fueron muy divertidos, pusieron a prueba mis conocimientos y en algunos momentos mi paciencia. Creo que puedo decir que a nivel personal, me siento satisfecho, cuando comencé, pensé que no podría ni resolver el primero, pero para mi sorpresa, pude completar todos los desafíos.

Se aplicaron conocimientos de distintas herramientas como IDA PRO (que hasta la fecha si lo había usado para algunas cosas pero no al nivel que requerí para esta serie de desafíos.

Pude obtener más confianza en desarrollar scripts (que habitualmente no realizo por diversos motivos) Cada uno de los desafíos me hizo pensar de forma distinta, pero también fue un reflejo del conocimiento actual que actualmente poseo, y que requiero más conocimientos para seguir avanzando en el mundo de ingeniería inversa.

Scripts

Challenge2-BinaryGeckoENG.py

```
Challenge2-BinaryGeckoENG.py x Ejercicio16.txt
AprendiendoReversing > Challenge2-BinaryGeckoENG.py > ...
1 user_name = input("Insert your name: ")
2 intermediate_key = []
3 for char in user_name:
4     new_char_code = ord(char) + 3
5     new_char = chr(new_char_code)
6     intermediate_key.append(new_char)
7 ciphered_string = "".join(intermediate_key)
8 final_key = ciphered_string[::-1]
9 print("\n--- Result ---")
10 print(f"Usuario: {user_name}")
11 print(f"KeyGen: {final_key}")
12 print("-----")

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\EntornoDesarrollo\MisProgramas\AprendiendoReversing> & C:/EntornoDesarrollo/MisProgramas/AprendiendoReversing/Challenge2-BinaryGeckoENG.py
Insert your name: solid

--- Result ---
Usuario: solid
KeyGen: glorv
-----

PS C:\EntornoDesarrollo\MisProgramas\AprendiendoReversing>
```

Nota: En realidad es la segunda versión que hice, porque la primera no contemplaba la sensibilidad de mayúsculas y minúsculas - si el nombre decía "Karasu" y proporcionaba una clave errónea, así que eso fue corregido:

Dejo algunas pruebas realizadas:

```
rollo/MisProgramas/Aprendiendo
Insert your name: Ricardo

--- Result ---
Usuario: Ricardo
KeyGen: rgudflU
-----
```

```
rt=== Level 2 ===
rtGoal: Get the correct key for your name. (Optional write a keygen)
rtEnter your name: Ricardo
rtEnter the generated key: rgudflU
rt[+] Correct! Generated key: rgudflU
```

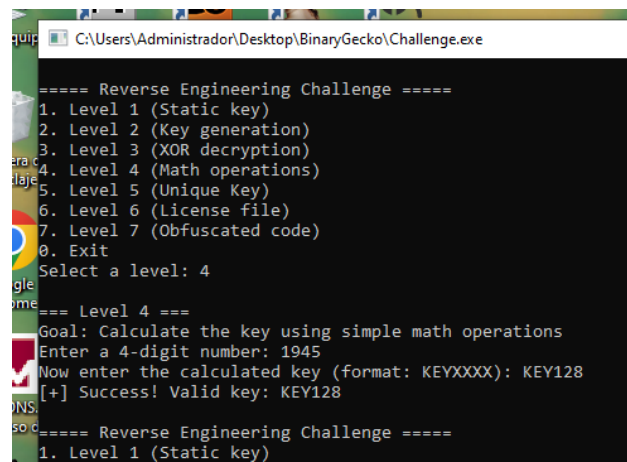
```
rollo/MisProgramas/Aprendiendo
Insert your name: SoLiD

--- Result ---
Usuario: SoLiD
KeyGen: GLoRV
```

```
rt=== Level 2 ===
rtGoal: Get the correct key for your name. (Optional write a keygen)
rtEnter your name: SoLiD
rtEnter the generated key: GLoRV
rt[+] Correct! Generated key: GLoRV
```

Challenge4-BinaryGeckoENG.py

```
1 valid_entry = False
2 while not valid_entry:
3     entrynum = input("Enter a 4-digit number (ej: 1933): ")
4     if len(entrynum) == 4 and entrynum.isdigit():
5         valid_entry = True
6     else:
7         print("Error: Please enter exactly 4 digits. Please try again.")
8 pair1_str = entrynum[0:2]
9 pair2_str = entrynum[2:4]
10 pair1 = int(pair1_str)
11 pair2 = int(pair2_str)
12 sum = pair1 + pair2
13 keyfinal = sum * 2
14 keyfinal = "KEY" + str(keyfinal)
15 print("\n--- CALCULATION DETAILS ---")
16 print(f"Number entered: {entrynum}")
17 print(f" - First Pair: {pair1}")
18 print(f" - Second Pair: {pair2}")
19 print(f" - Addition: {pair1} + {pair2} = {sum}")
20 print(f" - Result: {sum} x 2 = {keyfinal}")
21 print(f"\nThe generated key is: {keyfinal}")
```



```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 4

===== Level 4 =====
Goal: Calculate the key using simple math operations
Enter a 4-digit number: 1945
Now enter the calculated key (format: KEYXXXX): KEY128
[+] Success! Valid key: KEY128

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
```

```

Challenge2-BinaryGeckoENG.py X Challenge4-BinaryGeckoENG.py U X
AprendiendoReversing > Challenge4-BinaryGeckoENG.py > ...
6         else:
7             print("Error: Please enter exactly 4 digits. Plea
8
9         pair1_str = entrynum[0:2]
10        pair2_str = entrynum[2:4]
11        pair1 = int(pair1_str)
12        pair2 = int(pair2_str)
13        sum = pair1 + pair2
14        keyfinal = sum * 2
15        keyfinal = "KEY" + str(keyfinal)
16
17        print("\n--- CALCULATION DETAILS ---")
18        print(f"Number entered: {entrynum}")
19        print(f" - First Pair: {pair1}")
20        print(f" - Second Pair: {pair2}")
21        print(f" - Addition: {pair1} + {pair2} = {sum}")
22        print(f" - Result: {sum} x 2 = {keyfinal}")
23        print(f"\nThe generated key is: {keyfinal}")

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\EntornoDesarrollo\MisProgramas> & C:/EntornoDesarrollo/Python/Scripts/Python.exe C:\EntornoDesarrollo\MisProgramas/AprendiendoReversing/Challenge4-BinaryGeckoENG.py

Enter a 4-digit number (ej: 1933): 1945

```

--- CALCULATION DETAILS ---
Number entered: 1945
- First Pair: 19
- Second Pair: 45
- Addition: 19 + 45 = 64
- Result: 64 x 2 = KEY128

The generated key is: KEY128

```

Otras pruebas realizadas

```

16 print(f"Number entered: {entrynum}")
17 print(f" - First Pair: {pair1}")
18 print(f" - Second Pair: {pair2}")
19 print(f" - Addition: {pair1} + {pair2} = {sum}")
20 print(f" - Result: {sum} x 2 = {keyfinal}")
21 print(f"\nThe generated key is: {keyfinal}")

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\EntornoDesarrollo\MisProgramas> & C:/EntornoDesarrollo/Python/Scripts/Python.exe C:\EntornoDesarrollo\MisProgramas/AprendiendoReversing/Challenge4-BinaryGeckoENG.py

Enter a 4-digit number (ej: 1933): 9898

```

--- CALCULATION DETAILS ---
Number entered: 9898
- First Pair: 98
- Second Pair: 98
- Addition: 98 + 98 = 196
- Result: 196 x 2 = KEY392

The generated key is: KEY392
PS C:\EntornoDesarrollo\MisProgramas>

```

C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

Now enter the calculated key (format: KEYXXXX): KEY128
[+] Success! Valid key: KEY128

```

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 4

=== Level 4 ===
Goal: Calculate the key using simple math operations
Enter a 4-digit number: 9898
Now enter the calculated key (format: KEYXXXX): KEY392
[+] Success! Valid key: KEY392

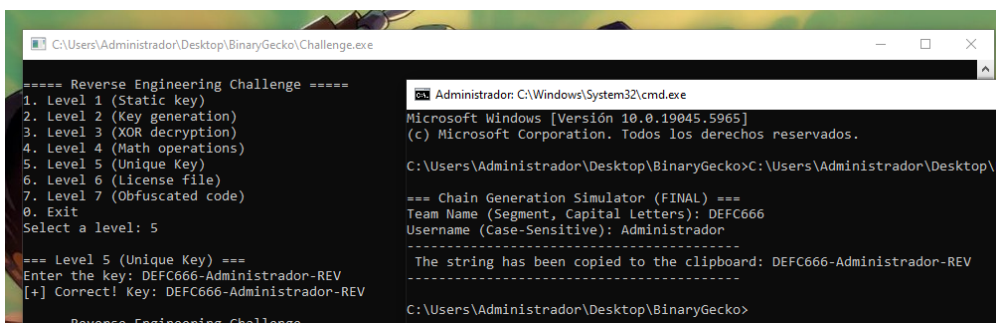
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level:

```

Challenge5-BinaryGeckoENG.py

```
Challenge2-BinaryGeckoENG.py Challenge4-BinaryGeckoENG.py Challenge5-BinaryGeckoENG.py U X
AprendiendoReversing > Challenge5-BinaryGeckoENG.py > ...
1 import socket
2 import getpass
3 import pyperclip
4
5 namepccomplete = socket.gethostname()
6
7 if '-' in namepccomplete:
8     namesegment = namepccomplete.split('-')[1]
9 else:
10    namesegment = namepccomplete
11    namesegment = namesegment.upper()
12 username = getpass.getuser()
13 basestrings = namesegment + "-" + username + "-REV"
14 try:
15     pyperclip.copy(basestrings)
16     print("\n=== Chain Generation Simulator (FINAL) ===")
17     print(f"Team Name (Segment, Capital Letters): {namesegment}")
18     print(f"Username (Case-Sensitive): {username}")
19     print("-----")
20     print(f"The string has been copied to the clipboard: {basestrings}")
21     print("-----")
22 except pyperclip.PyperclipException as e:
23
24     print("\nERROR: Could not access clipboard.")
25     print("Make sure you have a graphical environment or the necessary dependencies (e.g. xclip/xsel on Linux).")
26     print(f"The generated string is: {basestrings}")
27     print(f"Error details: {e}")
```

Yo sé que quien/es lean este documento, saben infinitamente más que yo, pero por las dudas aclaro que para que el script funcione debe tener instalado el pyperclip – Desde CMD(Windows)) -> pip install pyperclip (esto es para que al generarme el nombre de la pc y lo concatene y arme la clave correspondiente, quede en el portapapeles.



```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 5

=== Level 5 (Unique Key) ===
Enter the key: DEFC666-Administrador-REV
[+] Correct! Key: DEFC666-Administrador-REV
===== Reverse Engineering Challenge =====

C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.19045.5965]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Administrador\Desktop\BinaryGecko>C:\Users\Administrador\Desktop\B

=== Chain Generation Simulator (FINAL) ===
Team Name (Segment, Capital Letters): DEFC666
Username (Case-Sensitive): Administrador
-----
The string has been copied to the clipboard: DEFC666-Administrador-REV
-----

C:\Users\Administrador\Desktop\BinaryGecko>
```

Challenge6-BinaryGeckoENG.py

```
import os
import struct
import sys
destination_directory = r"C:\Users\Administrador\Desktop\BinaryGecko"
filename = "licencia.enc"
pathcomplete = os.path.join(destination_directory, filename)
head = struct.pack("<I", 1279869765)
substring1 = b"BinaryGecko\x00"
stuffed = b"\x00\x00\x00\x00"
substring2 = b"9898\x00"
final = head + substring1 + stuffed + substring2

try:
    os.makedirs(destination_directory, exist_ok=True)
    with open(pathcomplete, "wb") as f:
        f.write(final)

    print(f"    Success! File '{filename}' generated.")
    print(f"    Size: {len(final)} bytes")
    print(f"    Save path: {pathcomplete}")

except PermissionError:
    print(f"    Permissions error: Could not write to path '{destination_directory}'.", file=sys.stderr)
    print("    Make sure you run the script with sufficient permissions.", file=sys.stderr)
except Exception as e:
    print(f"    Unexpected error while writing file: {e}", file=sys.stderr)
```